CITYPULSE AI

DEEP RESEARCH & MASTER ENGINEERING BLUEPRINT

A source-grounded, local-first urban infrastructure intelligence MVP

Research cut-off: 14 October 2026
Proposed first pilot: one Mumbai area / ward, configurable after source verification
Audience: product owner + Claude Code Pro implementation workflow

MASTER REFERENCE • VERSION 1.0 • ZERO PAID RUNTIME API DEPENDENCIES

Document control and use

Field	Value
Purpose	Give Claude Code and the developer a shared, detailed view of the product, its constraints, its data risks and the intended phased execution.
Research date	14 October 2026. Official websites, source terms, catalogues and model licences must be rechecked before implementation or release.
Pilot geography	Mumbai is a working hypothesis, not a hard-coded dependency. The actual first ward/area is chosen only after data, boundary and user-access checks.
Stack policy	No framework is mandated. Claude Code first performs a strictly read-only repository audit, then proposes an evidence-based stack. User approval is required before code changes.
Runtime cost policy	No paid inference API or cloud credit is required for the core MVP. Local model weights and public datasets may require download/storage; public services have terms and rate limits.
Status convention	Verified fact, candidate source, design recommendation, assumption or unresolved question must be labelled separately.
Git workflow	Claude Code must not commit or push. It provides commands only; developer operates Git and approves phase gates.

Critical rule Never let document length or an AI-generated architecture replace actual repository inspection, source verification, test evidence and human approval.

How to use this blueprint

1. Read the executive section and the source-risk matrix first; the biggest project risk is data availability and representativeness, not model choice.
2. Open the separate Claude Code prompt pack for ready-to-paste, gated prompts. Prompt 0 is read-only and must be the first command set.
3. After the audit, ask Claude to propose a tech stack with options, tradeoffs, licensing, MacBook M1 constraints and zero-paid-runtime-API compliance.
4. Only after approving the architecture should implementation begin, in 3-4-task batches with tests and a clear stop point.
5. Use this blueprint as a decision aid, not permission to implement every optional idea. The MVP must still work if live civic feeds and all models are absent.

Executive brief

CityPulse AI is a local-first urban evidence workbench. It consolidates scattered reports and public context into auditable analysis of where and when civic problems are reported, which patterns recur, what sources support the interpretation, and what a human reviewer may want to investigate next. The initial wedge is evidence synthesis and recurrence analysis; forecasting and generative AI are optional layers that must prove their value.

The recommended first demonstration focuses on one issue category and one configurable pilot geography, potentially in Mumbai. It combines lawful public or user-authorised data, explicit source metadata, validated observations, deterministic summaries, a map/timeline, related-report candidates, cited policy/document retrieval and a draft action brief. If the target geography has no usable structured records, the product should still demonstrate its workflow using clearly labelled synthetic fixtures and manual CSV/GeoJSON imports.

This is not an official municipal control system, emergency warning service, real-time flooding prediction engine or replacement for existing complaint channels. A report is not necessarily an incident; a hotspot is a statistic; a recommendation is a draft; a forecast is only as good as its labels, time coverage and evaluation. The system must never treat missing data as proof that no problem exists.

The strongest product thesis

Wedge Evidence-backed civic issue intelligence: source provenance + recurrence and spatial analysis + weather/document context + cited, human-reviewed briefing.

What the MVP should demonstrate

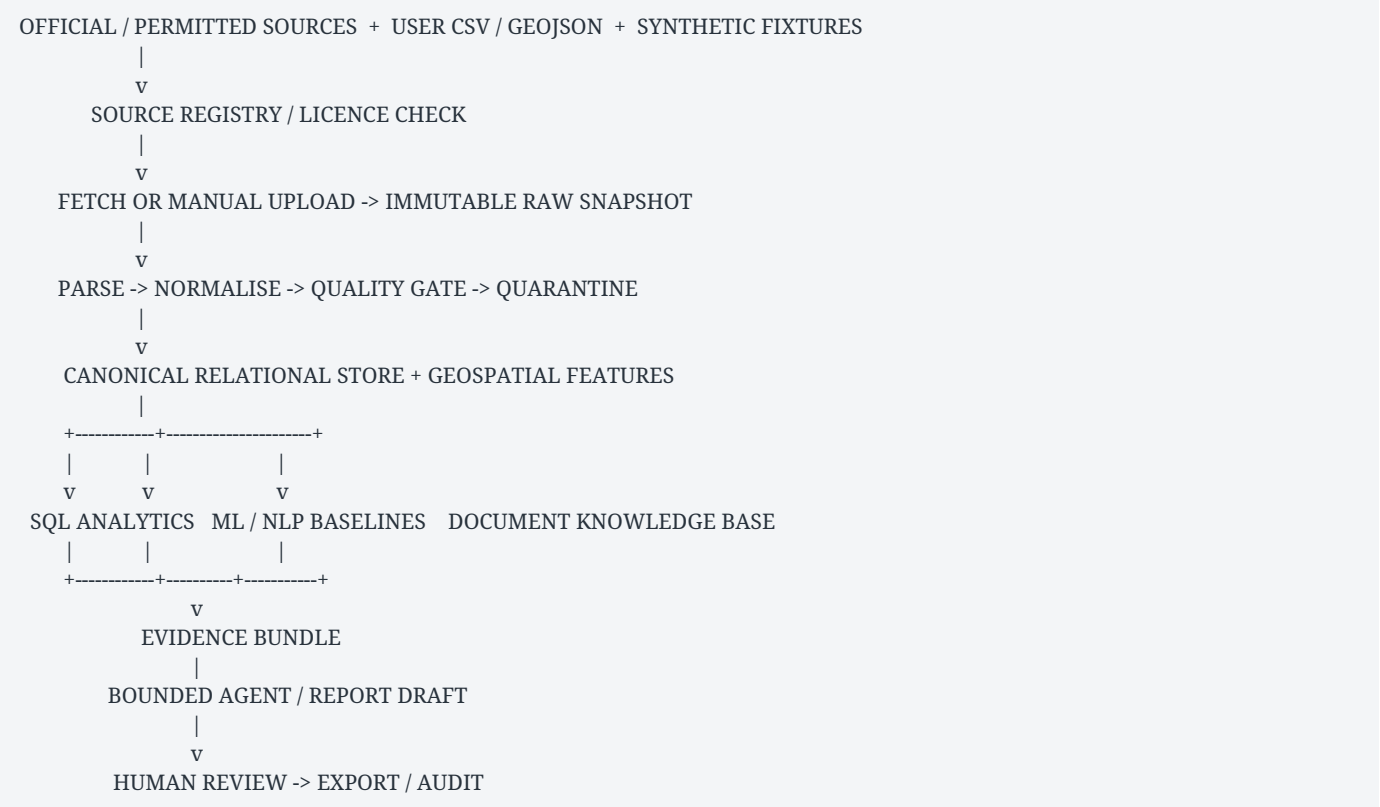
- Import a civic dataset with a visible schema preview and data-quality report.
- Trace every plotted or reported number back to source records and a documented definition.
- Show repeat locations and trends without conflating duplicate reports with distinct incidents.
- Bring in optional weather or hazard context with its real spatial/time resolution and caveats.
- Answer selected document questions with page/section citations and an explicit abstention when evidence is insufficient.
- Use a constrained analyst agent to query allowlisted analytical tools and draft a report for review, with no external side effects.
- Run core analytics locally with no paid inference API, and demonstrate in an offline/fixture mode.

Research conclusions: what is real, what is uncertain

Topic	Research finding	Design implication
Indian open data	OGD India is a discovery catalogue; individual records can have their own licence metadata and links to third-party sources. [R1-R3]	Verify the exact resource and its licence; keep import adapters optional.
Smart Cities portal	A large catalogue exists for participating Smart Cities. Its existence does not prove Mumbai coverage or a needed dataset/API. [R4]	Use as a candidate source catalogue, not a required live dependency.
Weather	IMD documents weather/rainfall/warning API families. [R5]	Verify endpoint behavior and timestamp semantics; use warnings only as cited context.
Local hazard context	MCGM disaster management plan may contain hazard context and mapped waterlogging-prone locations. [R6]	Potentially manual/map-derived, approximate and vintage-sensitive; not a live incident feed.
Satellite rainfall	NASA GPM IMERG has gridded precipitation data; approximately 0.1 degree (~10 km) at the cited product family. [R8]	Useful regional context; not street-level flood ground truth.
Map data	OSM data are reusable with attribution under ODbL; standard tile servers have separate usage constraints. [R11-R13]	Prefer regional extracts for analysis, show attribution, avoid bulk tile downloads.
Civic APIs	Open311 is an interoperability pattern; it does not establish that a target authority supports it. [R14]	Do not assume write access or automate official submissions.
Prior art	FixMyStreet is established open-source issue reporting. [R15]	Differentiate on traceable evidence synthesis and evaluated recurrence analytics, not reporting alone.
Trustworthy AI	OECD and NIST emphasize governance, transparency, accountability, safety and privacy for AI systems. [R18-R19]	Constrain agents and distinguish data facts from model interpretations.

Key uncertainty Actual availability, completeness, freshness, licensing and geographic resolution of the specific pilot datasets remain to be verified. No assumption of a municipal live API is made.

Reference architecture (technology-neutral)



Architectural principles: start with a modular monolith unless the audited repository has constraints requiring another pattern; keep raw input, canonical facts, analytical outputs and AI-generated drafts separate; make optional models replaceable; use typed tool schemas; preserve source lineage; and make the core path run without network services. A framework is not selected here by design.

Trust zones

Zone	Contains	Allowed behaviour	Never allow
A. Source intake	Downloads, uploads, external documents	Size-limited parse, checksum, source metadata	Executing macros, arbitrary path access, unbounded fetching
B. Canonical data	Validated observations and geography	Parameterized queries, audited updates	LLM direct writes, silent mutation of source facts
C. Derived analysis	Aggregates, cluster candidates, model runs	Versioned calculations and reproducible views	Unlabelled inference presented as fact
D. AI interface	Retrieved chunks, tool results, draft reports	Cite sources, bounded tools, abstain and escalate	Arbitrary SQL/shell/URLs or autonomous external actions

End-to-end data flow and traceability contract

Every result should be traceable across a stable chain of IDs. A source registry row describes the source and licence status. An ingestion batch describes a single attempt. A source snapshot or local file hash identifies the input version. A normalized observation links to its batch and carries quality flags. An analysis run identifies filters, metric version and the records contributing to a result. A report draft references analysis-run IDs and document chunk IDs. A review event records what the human accepted, edited or rejected.

Artifact	Required trace fields	Why it matters
Source	source_id, URL/file, owner, licence, reviewed_at	Makes data rights and provenance inspectable.
Batch	batch_id, source_id, started/finished, status, parser_version	Explains when and how a dataset entered the system.
Observation	observation_id, source/batch, timestamps, location method, quality flags	Separates the underlying record from model interpretation.
Analysis run	run_id, query/metric version, filters, time window, input IDs or aggregate snapshot	Makes counts and hotspots reproducible.
Model run	task, data version, model version, split, metrics, limitations	Prevents experimental predictions from masquerading as tested features.
Draft/review	draft_id, evidence refs, generated_at, reviewed_at, reviewer action	Keeps AI suggestions from being treated as official decisions.

Invariant No result in a public-facing report should exist without a scope, an observation window, source lineage, relevant quality flags and an explicit evidence-vs-inference distinction.

Implementation roadmap at a glance

Phase	Goal	Core outputs	Exit gate
0	Read-only repository audit	Inventory exact repo state, tests, contracts, risks; no changes	User approves audit/first architecture task.
1	Architecture and stack proposal	Decision records, data flow, schema/API draft, alternatives	User approves stack and first implementation batch.
2	Source register and fixtures	Verified/pending source matrix, offline fixture import	Repeatable import with clear data mode.
3	Schema and ingestion	Canonical schema, import preview, validation, quarantine, idempotency	Clean DB migration and import tests pass.
4	Deterministic analytics	Counts, trends, recurrence, related record explorer	Hand-calculated fixture values match.
5	Geospatial views	CRS checks, configurable boundaries, map/table fallback	Known coordinates and filters agree.
6	ML baseline	One chosen task, baseline, holdout, model card	Beats baseline or remains disabled/experimental.
7	NLP	Normalization, labels/place candidates, duplicates	Reviewed evaluation sample and privacy controls.
8	RAG	Small approved corpus, citations, no-answer benchmark	Citation and abstention tests pass.
9	Bounded agent	Typed tools, read-only SQL, step limits, review	Adversarial tests block unsafe tools.
10	UX and export	Core investigation flow and evidence brief	End-to-end task is usable.
11	Hardening	Security, licences, accessibility, evaluation	Critical release blockers closed.
12	Human pilot	One geography/problem/user task and measured feedback	Go/no-go memo with honest results.
13	Release gate	Runbook, known limits, reproducibility, costs	No unsupported claims; user explicitly closes phase.

Do not run all phases as one Claude Code request. Each phase may need several small batches; every batch should finish with changes, tests, evidence and an explicit stop. The roadmap is a dependency order, not a promise that every feature will be built or that a phase lasts a fixed number of days.

Table of contents and engineering-note index

A. Product, evidence and trust

001 CityPulse product thesis
002 Problem framing and measurable wedge
003 Prior art and differentiation
004 Target personas
005 Primary user journey
006 MVP scope and release definition
007 Non-goals and safety boundaries
008 Theory of change and value hypothesis
009 Product metrics and north-star
010 System context and trust boundaries
011 Research approach and evidence labels
012 Public data availability risk
013 Source register data model

B. Sources and source acceptance

014 OGD India catalogue workflow
015 Smart Cities Mission data portal
016 IMD weather and rainfall integration
017 MCGM disaster-management plan and hazard context
018 NASA GPM IMERG rainfall data
019 OpenStreetMap base geography
020 Copernicus and satellite-derived context
021 OSM usage policy and mapping UX
022 Open311 interoperability pattern
023 Source acceptance scoring

C. Canonical data model and ingestion

024 Data contracts and canonical model
025 Raw, staging, canonical and derived layers
026 Core entities and relationships
027 Observation and event semantics
028 Temporal storage and timestamp rules
029 Geospatial representation and CRS
030 Geometry and administrative boundaries
031 Place matching and geocoding
032 Data type, units and null semantics
033 Data quality framework
034 CSV/Excel import pipeline
035 GeoJSON and vector import
036 PDF and map extraction strategy
037 Duplicate import and idempotency
038 Quarantine, repair and reprocess
039 ETL orchestration and source adapters
040 API discovery and connector policy
041 Data freshness and staleness

D. Analytics and evidence briefs

042 Baseline descriptive analytics
043 Recurrence and hotspot definition
044 Spatial clustering options
045 Temporal trend and seasonality
046 Exposure-normalized comparisons
047 Rainfall-context analysis
048 Evidence-first briefing
049 SQL metric layer
050 Dashboard information architecture
051 Data lineage in the UI
052 Decision-support recommendation logic

E. ML evaluation and forecasting limits

053 ML scope selection and model card
054 Complaint category classification
055 Duplicate-report detection
056 Time-series forecasting of report volume
057 Flooding risk: what can and cannot be predicted
058 Anomaly detection and explainable flags
059 Deep learning decision gate
060 Computer vision as optional extension
061 Feature engineering and leakage prevention
062 ML evaluation and calibration
063 Fairness and coverage bias

F. NLP and text processing

064 NLP pipeline and normalization
065 Multilingual and code-mixed civic text
066 Named entity and place extraction
067 Text embeddings and semantic similarity
068 NLP privacy redaction

G. RAG and document evidence

069 RAG use case and evidence policy
070 Document ingestion for knowledge base
071 Chunking and retrieval strategy
072 RAG answer generation and citations
073 RAG evaluation framework
074 Prompt injection and retrieved document safety
075 RAG source conflict and freshness
076 Local LLM and zero paid API policy

H. Agentic AI and safe tools

077 Agent role and constrained autonomy
078 Agent planning and state machine
079 Read-only SQL tools for the agent
080 Human review and approval model
081 Agent tool authorization
082 Agent observability and replay

I. API, security, operations and runtime

083 API boundary and error design
084 Authentication, roles and local-first assumption
085 Upload security and parser hardening
086 Privacy and personal data
087 Security threat model
088 API security checklist
089 Open-source and model licence management
090 Environment and dependency reproducibility
091 Configuration and secrets
092 Local storage, backups and migration
093 Model/provider abstraction
094 Observability and application logs
095 Performance budgets for M1 laptop
096 Zero paid API cost architecture
097 Deployment scenarios and decision gate
098 Testing strategy and pyramid
099 Synthetic fixtures and safe demo mode
100 CI, release and code quality

J. Accessibility, pilot and startup validation

101 Accessibility and inclusive design
102 Pilot design and user research
103 Startup discovery and differentiation test
104 Portfolio narrative and honest claims

K. Phase-by-phase implementation plan

105 Phase 0: strictly read-only repository audit
106 Phase 1: architecture and stack decision

107 Phase 2: source inventory and fixtures
108 Phase 3: schema and ingestion
109 Phase 4: deterministic analytics
110 Phase 5: mapping and geospatial quality
111 Phase 6: ML baselines
112 Phase 7: NLP enhancements
113 Phase 8: RAG knowledge base
114 Phase 9: bounded agent workflow
115 Phase 10: UX and exports
116 Phase 11: hardening and evaluation
117 Phase 12: pilot and decision
118 Phase 13: roadmap exit criteria

L. Claude Code workflow, gates and costs

119 Claude Code working agreement
120 README and project guidance
121 Prompt design for implementation batches
122 Code review and phase gate
123 Cost governance and resource budget
124 Research uncertainty and source refresh

M. Reference architecture, schemas and release contracts

125 Reference architecture and data flow
126 Concrete database schema decision guide
127 Suggested observation record contract
128 Document, chunk and model-run contracts
129 API endpoint catalogue for an MVP
130 API errors, pagination and consistency
131 Source register starting matrix
132 Technical decision records template
133 Definition-of-done rubric

The engineering notes are intended as independent reviewable modules, not as 133 mandatory implementation tasks. Several notes may be closed together in one approved batch; complex phases should be split further as needed.

ENGINEERING NOTE 001 |

CityPulse product thesis

Purpose and product decision. CityPulse should make scattered urban evidence easier to inspect and act on. Its first value is not autonomous city management; it is a trustworthy chain from source record to mapped pattern to reviewed recommendation. The product is an evidence workbench for civic teams, researchers, resident associations and NGOs.

Implementation contract. The main interface should answer four questions: what is reported, where/when it happens, what is known from independent evidence, and what next step merits human review. Every visual or narrative summary should carry a source timestamp, geographic scope, completeness statement and confidence/quality indicator.

Failure modes and evidence constraints. A slick map with weak data is not intelligence. Avoid positioning as an official municipal system, emergency warning service or replacement for existing grievance channels. Do not claim a proprietary invention merely because AI is used.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to citypulse product thesis. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The opening demo must show a real dataset's limitations, one repeatable analysis, and how provenance affects interpretation. Product copy must distinguish observed reports from inferred risk.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R15], [R16], [R18]

ENGINEERING NOTE 002 |

Problem framing and measurable wedge

Purpose and product decision. The first problem slice is recurring civic friction: reports and public context are dispersed across formats, time periods, and geographic units. Reviewers spend time normalising evidence, spotting repeated locations, and composing a defensible summary. CityPulse should reduce that synthesis cost without inventing missing facts.

Implementation contract. Begin with one problem class such as waterlogging or road damage, one locally selected pilot geography, and a small set of source types. Capture every dataset's unit of observation, timestamps, spatial resolution and known blind spots. Expand only when the first wedge delivers repeatable value.

Failure modes and evidence constraints. The product cannot responsibly estimate an event probability without historical labelled events, coverage checks and independent validation. A city-wide scope may be too broad for a solo developer if the only reliable data are PDFs or aggregate rainfall.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to problem framing and measurable wedge. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Write a one-page product charter with user, decision, input evidence, output, non-goals, metric and validation plan. A reviewer unfamiliar with the project should understand the wedge in two minutes.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R6], [R7], [R17]

ENGINEERING NOTE 003 |

Prior art and differentiation

Purpose and product decision. Civic reporting itself is established prior art. FixMyStreet provides a map-based public problem-reporting platform and Open311 describes a common civic service-request API pattern. CityPulse should differentiate through evidence provenance, recurrence analytics, cross-source context, careful uncertainty and human-reviewed action briefs.

Implementation contract. Make a comparison matrix for reporting, issue routing, public dashboards, GIS/remote sensing and AI assistants. Compare source transparency, deduplication, recurrence, spatial-temporal evidence, uncertainty, local installability, language support and evaluation. Only claim gaps that have been confirmed by current product review.

Failure modes and evidence constraints. Do not say no one has built this before. Existing municipal command centres, complaint systems and private GIS providers may overlap in features even if their data are not public. Novelty should be framed as a focused, reproducible implementation for an underserved workflow.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to prior art and differentiation. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Ship a prior-art register with URL, review date, features observed, evidence excerpt, limitations and differentiation decision. Have someone outside the team challenge the claims.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R14], [R15], [R18]

ENGINEERING NOTE 004 |

Target personas

Purpose and product decision. The first users are likely analyst-like roles rather than the entire public: a civic researcher, ward-level NGO coordinator, residents-association volunteer or journalist who wants defensible summaries. A municipal officer is a possible stakeholder but should not be assumed to adopt the tool.

Implementation contract. Create persona-specific tasks, not stereotypes. For each persona define the decision they own, data they can lawfully access, time constraint, acceptable error, trust needs and their current workaround. Observe real users before treating a persona as validated.

Failure modes and evidence constraints. Avoid implying that citizens' reports are representative of all residents. Digital access, language, safety and willingness to report differ across groups; low complaint volume may indicate weak reporting rather than low need.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to target personas. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Run five short problem interviews if feasible, log anonymised observations, and record which assumptions changed. If interview access is not possible, label the persona as a hypothesis.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R16], [R17], [R18]

ENGINEERING NOTE 005 |

Primary user journey

Purpose and product decision. A first journey should be simple: choose area and timeframe, inspect source/data quality, open a hotspot, see related reports and rainfall context, inspect policy evidence, and export a human-reviewed action brief. Every step must preserve a path back to raw records.

Implementation contract. The UI should keep observed facts, computed statistics, model outputs and LLM-generated explanations visually distinct. Use drill-through paths and source links. Let the analyst inspect rows, not just a chart. Provide empty states when no evidence is available.

Failure modes and evidence constraints. A fluent LLM summary can create false certainty. The user should never have to trust an uncited paragraph when the underlying rows or time window can be inspected.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to primary user journey. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A usability session should let a first-time user answer who/what/where/when, what is unknown and why a recommendation was made without developer help. Record task completion and confusion points.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19]

ENGINEERING NOTE 006 |

MVP scope and release definition

Purpose and product decision. The MVP is a local-first civic evidence workbench, not a fully integrated smart-city control platform. A credible release includes source catalogue, file import, validated canonical records, SQL-backed filters, map/timeline views, deterministic hotspot summaries, basic duplicate detection, cited document retrieval, safe read-only agent actions and a report export.

Implementation contract. Keep data connectors pluggable. Support CSV/GeoJSON first and PDF-derived data only when a human verifies extraction. Build an explicit demo-data mode. ML features can be feature-flagged and labelled experimental until their evaluation gates pass.

Failure modes and evidence constraints. Do not make login/roles, multi-tenancy, live feeds, mobile app, field-officer dispatch, citizen messaging or municipality write-back prerequisites for the first demo. Introduce them only with evidence and threat modelling.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to mvp scope and release definition. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The MVP can be run from a clean checkout on the developer's Mac without a paid key, has one end-to-end documented scenario, and all major outputs trace back to input records.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R5], [R15]

ENGINEERING NOTE 007 |

Non-goals and safety boundaries

Purpose and product decision. CityPulse is neither an emergency dispatch system nor the official source of weather warnings. It must not auto-submit complaints, contact government departments, publish accusations about named people, direct emergency response, or claim an area is safe because data is missing.

Implementation contract. Render official warnings as sourced information with issued-at and validity fields. For critical decisions, link back to official channels and explain that CityPulse is an analytical tool. A proposed task stays a draft until a human explicitly approves it; the MVP should not provide an external-action mechanism.

Failure modes and evidence constraints. Do not ingest sensitive personal details unless necessary and justified. A civic complaint can expose addresses, phone numbers, health concerns or allegations. Prefer public aggregate and synthetic records; redact imported personal data before indexing.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to non-goals and safety boundaries. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Write an explicit prohibited-actions list into the product docs, API tests and agent system design. Any unsafe action should fail closed and leave a redacted audit event.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19], [R20]

ENGINEERING NOTE 008 |

Theory of change and value hypothesis

Purpose and product decision. The hypothesised value chain is: normalised records reduce data-preparation effort; recurrence and spatial summaries help analysts prioritise investigation; citations improve trust; reviewed action briefs make it easier to coordinate follow-up. Each link must be measured, not assumed.

Implementation contract. Define baseline effort such as minutes to find related reports and prepare a weekly summary. Compare with the assisted workflow using the same input set. Separately measure precision of duplicate suggestions, source-link validity and the fraction of statements verified by reviewers.

Failure modes and evidence constraints. A hotspot score is not a direct estimate of harm or a moral ranking of neighbourhoods. Reporting density reflects exposure, access, attention and data coverage. Use labels like “high observed reporting volume” rather than “most neglected” unless independently established.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to theory of change and value hypothesis. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Produce a validation protocol before tuning the system. Include success thresholds agreed with a pilot reviewer and a plan for measuring false prioritisation.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R17], [R18], [R19]

ENGINEERING NOTE 009 |

Product metrics and north-star

Purpose and product decision. The north-star should measure evidence-backed analyst work completed, not messages exchanged with an AI assistant. A useful unit is a reviewed area-time brief with source citations, reproducible statistics and recorded reviewer edits.

Implementation contract. Track successful imports, quarantined rows, source freshness, geocoding coverage, duplicate-review acceptance, report-generation time, citation verification, model calibration where applicable, agent tool errors and task completion. Separate product metrics from model metrics.

Failure modes and evidence constraints. Avoid vanity metrics such as total records, model complexity or number of agents. More records may simply mean duplicate or noisy collection. Engagement without decision quality does not prove value.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to product metrics and north-star. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Create a metric dictionary containing formula, event source, denominator, owner, interpretation, limitations and target. Any KPI must be testable or observable from event logs, not imagined.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19]

ENGINEERING NOTE 010 |

System context and trust boundaries

Purpose and product decision. Treat CityPulse as a local data platform with four trust zones: original source files, normalised data store, derived analytical outputs, and AI interfaces. A retrieval system can read untrusted documents; an agent can invoke only allowlisted local tools; a human approves any consequential output.

Implementation contract. Label data and derived values with provenance metadata. A user's uploaded source is not automatically trustworthy merely because it is in the system. Parse files in isolated, size-limited flows, treat text as data rather than instructions and avoid letting document content modify the agent's policy.

Failure modes and evidence constraints. Threats include malformed CSVs, zip bombs, malicious PDFs, prompt injection in public reports, SQL injection, leaking contact details, stale warnings and model outputs mistaken for verified fact. The architecture should constrain blast radius even when components fail.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to system context and trust boundaries. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Create a one-page trust-boundary diagram, a threat register and an approval matrix before implementing the agent. Every crossing of a boundary must have explicit validation and logging.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 011 |

Research approach and evidence labels

Purpose and product decision. The blueprint distinguishes verified official facts, candidate sources, design recommendations, assumptions and unresolved questions. A source's existence does not prove it has the required fields, geographic detail, licence or current accessibility.

Implementation contract. Every connector candidate gets a source card with official URL, access method, data owner, file/API shape, observed-at date, licence text, attribution, update expectation, spatial/temporal granularity, authentication, rate limits and validation status. Keep a captured sample and checksum when permitted.

Failure modes and evidence constraints. Websites and API docs change. The build pipeline must verify the live source metadata before depending on it. If access cannot be checked from the current environment, record “verification pending” instead of fabricating a successful result.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to research approach and evidence labels. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Before adding an adapter, show the user the source evidence and a sample record. If a portal lacks a needed dataset, proceed with import templates and approved synthetic data rather than blocking the MVP.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2], [R4], [R5]

ENGINEERING NOTE 012 |

Public data availability risk

Purpose and product decision. Urban data availability is fragmented across municipal departments, open-data catalogues, weather services, maps, disaster plans and research datasets. Some source pages are catalogues or PDFs rather than machine-readable feeds. A zero-API budget does not remove access, licensing or freshness constraints.

Implementation contract. Use a source portfolio, not a single dependency: stable static boundary/base layers, official weather context when conditions permit, hand-entered or consented incident observations, and synthetic fixtures for repeatable tests. Make each source optional and expose its status in the UI.

Failure modes and evidence constraints. Never silently replace missing live data with synthetic rows. Tag records with source type and display a visible demo-data banner. A connector returning no records must not be interpreted as no incidents.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to public data availability risk. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Source status page must show last attempt, last success, record count, schema version, licence review date and reason for blocked/disabled status. The app remains useful offline.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R4], [R5]

ENGINEERING NOTE 013 |

Source register data model

Purpose and product decision. A source register makes provenance operational rather than decorative. Store a stable source identifier and a human-readable citation so each plotted result and report can answer which input records support it.

Implementation contract. Suggested fields: source_id, source_name, owner, canonical_url, access_method, licence_name, licence_url, attribution_text, terms_checked_at, source_status, geographic_coverage, temporal_coverage, update_expectation, schema_version, parser_version, last_attempt_at, last_success_at, data_contact and known_limitations. Keep source metadata versioned.

Failure modes and evidence constraints. Don't hard-code source claims in UI components or rely on a homepage as the data licence. Dataset-level licences and linked third-party material may differ.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to source register data model. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Tests should require a licence or explicit “pending review” state before distribution. Every ingested batch links to one source record and a source snapshot/checksum where lawful.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2], [R3], [R25]

ENGINEERING NOTE 014 |

OGD India catalogue workflow

Purpose and product decision. The OGD India platform is a starting point for discovering datasets, but its own terms warn that catalogues can link to other sources and the individual resource's license metadata matters. Data.gov.in also includes dataset/web-service resources; discovery must be performed for the particular topic and geography.

Implementation contract. Build a manual source discovery worksheet first. Search relevant terms such as rainfall, drainage, wards, waterlogging, complaints, roads and public works; inspect records; download one resource; inspect fields and dates; save metadata and licence references. Only then decide whether to code an adapter.

Failure modes and evidence constraints. Avoid assuming all records use the same licence, are current, or are machine-readable. Catalog metadata may not match the actual file, and a public URL may be temporarily unavailable.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to ogd india catalogue workflow. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A checked source record includes a saved metadata snapshot, source URL, license evidence and an example file or a reason the source failed verification.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2], [R3], [R25]

ENGINEERING NOTE 015 |

Smart Cities Mission data portal

Purpose and product decision. The Smart Cities data portal offers catalogues for participating cities. It is useful for benchmarking and finding examples of civic-data publishing, but its city list does not imply Mumbai coverage or a particular live feed.

Implementation contract. Search by city and domain. Record whether an item is actually related to the target area, its publication date, data fields, licence and API/download mechanism. Use it as a candidate source, not an architectural dependency. If a suitable city dataset exists elsewhere, keep the connector configurable.

Failure modes and evidence constraints. Dataset counts are not equivalent to usable urban-infrastructure coverage. The portal may contain unrelated records, static tables or stale observations.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to smart cities mission data portal. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A source review log demonstrates one inspected dataset and a yes/no/unknown decision with rationale. No code should pretend that a city adapter exists until a sample has been tested.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R4]

ENGINEERING NOTE 016 |

IMD weather and rainfall integration

Purpose and product decision. The IMD public API reference lists city forecasts, current weather, district warnings and rainfall endpoints. These may offer official weather context for the region, but endpoint semantics, update timing, access conditions and operational guarantees must be rechecked during implementation.

Implementation contract. Implement a narrow client behind an interface. Store requested endpoint, parameters, request time, status, observation time, issue time, valid period, unit, raw payload hash and normalised fields. Separate observation time from fetch time. Preserve warning code and original text for audit.

Failure modes and evidence constraints. District-level rainfall and weather warnings cannot determine whether one road is flooded. A missing response is a connector failure, not a clear-weather signal. Do not issue alerts that compete with official IMD warnings.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to imd weather and rainfall integration. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A captured fixture test proves unit conversions, timestamp parsing, missing-value handling and warning mapping. Live calls are optional and must be behind explicit user-triggered sync or conservative scheduled polling.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R5]

ENGINEERING NOTE 017 |

MCGM disaster-management plan and hazard context

Purpose and product decision. The MCGM/Mumbai District Disaster Management Plan is a candidate official reference for local hazards and waterlogging-prone locations. A PDF map is contextual evidence, not necessarily a GIS-ready polygon or machine-readable incident archive.

Implementation contract. First inspect the plan's publication date, map legend, scale, boundary vintage and source notes. If a map must be traced or georeferenced, preserve the transformation method, control points, uncertainty and reviewer. Store derived features separately from source facts.

Failure modes and evidence constraints. A scanned map can be misread or misaligned. Do not turn approximate map marks into precise point truth. Historical known spots may not describe current drainage condition.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to mcgm disaster-management plan and hazard context. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Every manually derived layer has a provenance record, method, confidence/uncertainty, author/reviewer and visual QA screenshot. Display “digitised from map, approximate” when appropriate.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R6], [R7]

ENGINEERING NOTE 018 |

NASA GPM IMERG rainfall data

Purpose and product decision. NASA GPM IMERG provides gridded precipitation products with historical coverage and multiple product latencies. Its spatial resolution is approximately 0.1 degree or around 10 km, so it can help regional rainfall context but is not street-level flood ground truth.

Implementation contract. For a student MVP, use a downloaded public sample or official viewer/file distribution, pin the product version, variable, accumulation window, time standard and file checksum. Aggregate raster values to an explicitly named spatial unit and record the grid cell identifier.

Failure modes and evidence constraints. Satellite estimates can differ from gauges and local convective rainfall. Historical product reprocessing can change values across versions. Never present a grid-cell measurement as a reading at a specific street address.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to nasa gpm imerg rainfall data. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Document the resolution visibly, compare with any available gauge observations, and make the feature optional. A test must fail if a high-resolution map layer implies finer precision than the source supports.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R8]

ENGINEERING NOTE 019 |

OpenStreetMap base geography

Purpose and product decision. OSM may provide roads, paths, land-use and points of interest for local analysis. The data licence and service policies are separate concerns: downloading extracts is different from using public tile servers or editing APIs.

Implementation contract. Prefer a regional extract for local analytical work; cache and version the extract date, attribution and ODbL notes. Keep geometry source, OSM feature IDs and import timestamp. Use OSM base features for spatial context, not as authoritative municipal boundaries unless corroborated.

Failure modes and evidence constraints. OSM feature coverage and tagging quality vary. Public tile servers are best-effort and prohibit bulk tile prefetching. Do not treat “free data” as “unlimited free API calls.”

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to openstreetmap base geography. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Display OSM attribution. Use fixtures for tests and avoid repeated large queries to public services. The release checklist verifies ODbL obligations and map/tile provider terms.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R11], [R12], [R13]

ENGINEERING NOTE 020 |

Copernicus and satellite-derived context

Purpose and product decision. Copernicus Sentinel data are generally made available on a free, full and open basis under stated conditions; actual portal access and the relevant service/product terms still need to be checked. Satellite imagery may support land-cover or change context, but clouds, revisit, processing and spatial resolution limit the claim.

Implementation contract. Treat imagery as an optional research extension. Start with a known product, clear licence, downloaded sample and reproducible preprocessing. Save sensor/product, acquisition time, cloud mask, CRS, processing version and spatial resolution.

Failure modes and evidence constraints. A satellite image cannot automatically establish that trash, potholes or waterlogging were present at a particular hour. Avoid sophisticated imagery processing until there is a validated label set and a question it can answer.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to copernicus and satellite-derived context. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The first MVP should run without satellite images. Any later feature describes the exact measured proxy and validates it with independent references.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R9], [R10]

ENGINEERING NOTE 021 |

OSM usage policy and mapping UX

Purpose and product decision. OSM data are reusable with attribution under ODbL, but OSM Foundation tile services have strict usage policies, no SLA and restrictions on bulk downloading. A public tile URL is not a universal production map API.

Implementation contract. Make map provider configurable, include visible attribution, avoid bulk tile collection, follow caching guidance, and use an offline/static map view or self-hosted/local rendered layers for a zero-internet demo. Separate basemap provenance from incident data.

Failure modes and evidence constraints. A map disappearing because a third-party tile service is down must not make the analysis inaccessible. Geocoding services can have separate rate limits and privacy implications.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to osm usage policy and mapping ux. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Demonstrate the app with local GeoJSON and a fallback tabular view. Provider terms and contact information are listed in deployment documentation.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R11], [R12], [R13]

ENGINEERING NOTE 022 |

Open311 interoperability pattern

Purpose and product decision. Open311 describes a way for civic service requests to be exchanged through HTTP APIs. It is an integration pattern, not evidence of a live Open311-compatible municipal endpoint in the selected city.

Implementation contract. Use the concept to shape an internal normalized issue schema: service type, description, status, location, reported/updated timestamps and provider reference. If a real API is later found, map source fields to this schema through a connector-specific transform with loss reporting.

Failure modes and evidence constraints. Do not reverse-engineer undocumented systems, automate form submissions or imply that CityPulse can submit a complaint through Open311 without a verified authorised integration. Preserve unknown or unmapped fields.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to open311 interoperability pattern. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A schema mapping review documents all mapped, dropped and transformed fields; synthetic fixtures test pagination, duplicate IDs, invalid timestamps and partial API failures.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R14]

ENGINEERING NOTE 023 |

Source acceptance scoring

Purpose and product decision. A transparent source score prevents an exciting dataset from being used beyond its quality. Assess officialness, license clarity, coverage, freshness, granularity, machine readability, stability, bias and validation effort separately rather than compressing everything into a mysterious single number.

Implementation contract. Use categorical gates plus evidence notes: eligible, eligible with caveat, research only, blocked, or rejected. Hard gates include lawful use and intelligible provenance. Soft dimensions can be scored from 0-2 for prioritisation but retain the underlying rationale.

Failure modes and evidence constraints. A high score does not mean ground truth. Official data may be delayed or aggregated; crowdsourced data may be timely but unrepresentative. Do not let a weighted average erase a license or privacy blocker.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to source acceptance scoring. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Review at least five potential source families and document why the MVP includes or excludes each. The source selection decision is version-controlled as documentation after user approval.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2], [R4], [R5], [R8], [R11]

ENGINEERING NOTE 024 |

Data contracts and canonical model

Purpose and product decision. A canonical contract gives analysts, map views and models one consistent representation while preserving the original source semantics. Keep the original raw payload immutable where permitted and derive normalized records with explicit transformations.

Implementation contract. Use canonical fields for record ID, source ID, external ID, issue category, raw and cleaned text, observed/report/update timestamps, geometry or area identifier, temporal precision, status, severity if supplied, evidence kind, data quality flags and ingestion batch ID. Separate facts from computed features.

Failure modes and evidence constraints. Do not infer severity, resolution or location from absence. A record with only a ward name should not get a fake point coordinate. Keep raw values next to normalized values for audit.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to data contracts and canonical model. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A data dictionary defines each field, type, nullability, units, controlled vocabulary, source of truth, transformation and expected range. Schema validation rejects breaking changes clearly.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R5], [R14]

ENGINEERING NOTE 025 |

Raw, staging, canonical and derived layers

Purpose and product decision. A layered pipeline improves reproducibility. Raw stores what came from the source; staging parses and checks; canonical stores validated domain records; derived tables contain rollups, duplicate candidates, model scores and reports.

Implementation contract. Keep transformations idempotent. Re-importing the same batch should not silently create new copies. Record pipeline version and lineage between source object, parsed row, canonical record and derived output. A reprocess operation should produce comparable output for the same inputs and code version.

Failure modes and evidence constraints. Do not update raw source fields in place when normalizing. If parser behavior changes, create a new processing version and explain differences.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to raw, staging, canonical and derived layers. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A test runs the same fixture twice, checks idempotence, then deliberately changes parser version to ensure reprocessing is traceable.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2]

ENGINEERING NOTE 026 |

Core entities and relationships

Purpose and product decision. The relational model needs to represent time, space, sources and observations without conflating them. Entities should be small enough to query and auditable enough to explain derived results.

Implementation contract. Proposed entities: sources, source_snapshots, ingestion_batches, observations, places/areas, geometries, evidence_documents, document_chunks, duplicate_groups, model_runs, derived_metrics, analysis_runs, recommendation_drafts, approval_events and audit_events. For MVP, combine tables only where it simplifies without losing provenance.

Failure modes and evidence constraints. Avoid over-designing event-sourcing or multi-tenant architecture before the workflow exists. Yet do not collapse source, user-supplied record and model output into one giant JSON blob.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to core entities and relationships. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. An entity-relationship diagram names primary keys, foreign keys, cardinality, retention and sensitive-data flags. Migration tests create a blank database and upgrade from the current schema.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R23]

ENGINEERING NOTE 027 |

Observation and event semantics

Purpose and product decision. A complaint or incident report is an observation about a possible event, not necessarily the event itself. Multiple people can report one event; one report may describe many issues; one issue can persist across multiple dates.

Implementation contract. For MVP, use an observation record and a reviewed grouping for suspected same-event reports. Store `reported_at`, `observed_at`, `event_start_at`, `event_end_at` only when known, and a time precision field. A derived incident cluster must carry its algorithm version and reviewer status.

Failure modes and evidence constraints. Do not count all complaint rows as independent incidents. Missing start time should not be substituted with report submission time without a flag.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to observation and event semantics. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Analytics explicitly distinguishes report count, distinct location count, reviewed incident groups and unresolved report count. Tests cover repeated reports and uncertain timestamps.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R14], [R15]

ENGINEERING NOTE 028 |

Temporal storage and timestamp rules

Purpose and product decision. Time is central to recurrence and forecasting. Data sources may use UTC, local time, issue time, observation time, fetch time and validity windows; these cannot be interchanged.

Implementation contract. Store instants in a canonical timezone such as UTC and retain source timezone/offset when available. Convert to Asia/Kolkata only for display or local aggregation. Store date-only observations as dates rather than synthetic midnight instants. Keep `ingested_at` and `source_updated_at` separately.

Failure modes and evidence constraints. A daily rainfall field can refer to a past 24-hour window rather than a calendar day. Incorrect alignment creates false correlations or leakage.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to temporal storage and timestamp rules. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Test ambiguous dates, missing time zone, day boundaries, leap days, late-arriving records and daylight-saving behavior for any international source. Report temporal granularity in every analysis.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R5], [R8], [R21]

ENGINEERING NOTE 029 |

Geospatial representation and CRS

Purpose and product decision. Locations need coordinate reference system metadata. Latitude/longitude are angular coordinates; distance and buffer operations should use an appropriate projected CRS for the region, not naïve degree arithmetic.

Implementation contract. Store source coordinates as received and normalized geometry with CRS. For Mumbai-scale calculations, select a suitable local projected CRS and document the EPSG and rationale. Validate coordinate ranges, geometry validity and whether polygons are within the expected bounds.

Failure modes and evidence constraints. Do not guess coordinate order. GeoJSON commonly uses longitude then latitude; many user interfaces show latitude then longitude. A swapped point can land far from the intended city.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to geospatial representation and crs. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Unit tests cover known control points, invalid coordinates, polygons, out-of-bounds locations and CRS round-trips. Map output must display a location precision indicator.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R22], [R23]

ENGINEERING NOTE 030 |

Geometry and administrative boundaries

Purpose and product decision. Ward and neighbourhood boundaries change across time, source and administrative reform. A geographic join is only interpretable when the boundary version and time basis are known.

Implementation contract. Maintain boundary source, vintage, CRS, feature code, administrative level and validity period where available. If the city/ward boundary file cannot be verified, make the app accept GeoJSON and mark the uploaded version as user supplied. Do not ship a guessed ward polygon as fact.

Failure modes and evidence constraints. A named ward boundary from one source may not align with a map in a disaster plan. Boundaries from OSM or unofficial sites may be approximate.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to geometry and administrative boundaries. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A boundary QA report records feature count, invalid geometries, gaps/overlaps, CRS, bounding box and provenance; the UI shows the source and version.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R6], [R11], [R22]

ENGINEERING NOTE 031 |

Place matching and geocoding

Purpose and product decision. Many incident reports contain a locality, landmark or informal address instead of coordinates. Geocoding should preserve ambiguity instead of quietly forcing every description to a precise point.

Implementation contract. First try deterministic matching against known ward/locality names and alias tables. Next, offer a user-reviewed candidate list. Store the original location string, normalized candidate, method, coordinates, confidence and reviewer decision. External geocoding is optional and should not receive sensitive report text.

Failure modes and evidence constraints. Neighbourhood names can be shared, misspelled or informal; point coordinates are often absent. A low-confidence point must not be displayed with misleading precision.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to place matching and geocoding. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Measure geocoding coverage and precision on a manually labelled sample. Every result can be reverted and has an explicit `unresolved` state.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R11], [R22]

ENGINEERING NOTE 032 |

Data type, units and null semantics

Purpose and product decision. Normalisation should not erase meaning. Units, categorical codes, censored values and “not observed” must remain distinguishable from real zero.

Implementation contract. Use explicit numeric units, controlled vocabularies and per-field metadata. For example rainfall millimetres with accumulation window; temperatures in Celsius; counts as integers; coordinates with CRS; warning code as original code plus documented interpretation. Keep source units in raw/staging records.

Failure modes and evidence constraints. Zero rainfall and missing rainfall are not equivalent. A no-data IMD code must not turn into a zero; blank severity must not default to low.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to data type, units and null semantics. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Validation reports invalid range, unrecognized code, missing required field and conversion status; test vectors cover each failure and confirm no silent coercion.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R5], [R8]

ENGINEERING NOTE 033 |

Data quality framework

Purpose and product decision. Data quality is multidimensional: completeness, validity, consistency, uniqueness, timeliness, spatial precision and representativeness. A single percentage can conceal the dimensions that matter for a decision.

Implementation contract. For every batch calculate row count, accepted/quarantined count, field completeness, duplicate rates, timestamp range, coordinate bounds and source freshness. Keep row-level error codes and downloadable validation results. Let users filter analyses to high-quality rows.

Failure modes and evidence constraints. Do not silently discard records. Data quality flags should not automatically imply the underlying report is false; they indicate processing uncertainty.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to data quality framework. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. An import report is reproducible and test-covered. A batch with zero accepted records or suspiciously large volume fails visibly rather than producing a blank yet “successful” dashboard.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2]

ENGINEERING NOTE 034 |

CSV/Excel import pipeline

Purpose and product decision. Manual file import is the first dependable connector when APIs are absent or inaccessible. It is also useful for a pilot partner who can export a spreadsheet.

Implementation contract. Provide schema preview, delimiter/encoding detection, column mapping, date and coordinate parsing, duplicate detection, row limits and a dry-run summary before writing to canonical tables. Preserve original file name, hash, upload time and source attribution metadata.

Failure modes and evidence constraints. CSV content can contain formulas, unexpected large fields, inconsistent dates, invalid encodings and potentially private information. Spreadsheet formulas must not execute when exports are opened in spreadsheet software.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to csv/excel import pipeline. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Tests cover common encodings, separators, quoted commas, multiline text, oversized columns, malformed rows and formula-injection-safe exports. UI tells the user how many rows will be accepted or quarantined.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2]

ENGINEERING NOTE 035 |

GeoJSON and vector import

Purpose and product decision. GeoJSON is a useful low-friction way to import points and boundary polygons. Accept it as a format, not as proof its coordinate system, content or licence is correct.

Implementation contract. Validate geometry types, feature count, CRS assumption, property fields, bounding extent and invalid geometries. Because GeoJSON normally uses WGS84 longitude/latitude coordinates, document that assumption and allow preview before persist. Keep feature properties and source file hash.

Failure modes and evidence constraints. Malformed geometry may break spatial joins or cause performance issues. Do not render untrusted geometries without limits. A polygon with no metadata should not be assigned an official ward label.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to geojson and vector import. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Import is transactional: either validate and report a bounded set of issues or quarantine the whole batch where partial ingestion would be misleading.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R22], [R23]

ENGINEERING NOTE 036 |

PDF and map extraction strategy

Purpose and product decision. PDF is a last-resort ingestion path for contextual documents and scanned maps, not the default format for operational records. Text extraction can lose table structure; scanned maps require human verification and georeferencing.

Implementation contract. Record page references and extraction method. Extract tabular content to staging only with a row-level source page; use screenshots for map georeferencing and record control points/residual error. A human validates every transformed GIS feature before it enters the canonical store.

Failure modes and evidence constraints. Never present OCR output as a confirmed incident record without verification. Copyright and third-party map rights remain separate from the fact that the document can be viewed online.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to pdf and map extraction strategy. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A sample document test verifies page lineage, extraction confidence and visual review. Unsupported PDFs are rejected with an actionable reason rather than guessed.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R6], [R7]

ENGINEERING NOTE 037 |

Duplicate import and idempotency

Purpose and product decision. Repeat imports are common when users retry uploads or sync a source. Duplicates can inflate apparent issue volume and distort hotspots.

Implementation contract. Use a source-provided ID where available. Otherwise construct a stable fingerprint from normalized source ID, timestamp granularity, text hash and location bucket, but only as a candidate key. Keep both idempotency key and fuzzy duplicate suggestions separate.

Failure modes and evidence constraints. A fingerprint collision can merge legitimate reports. Identical text at two distinct locations or separate times should not be collapsed automatically.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to duplicate import and idempotency. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Repeated same-file import causes no extra canonical rows. Fuzzy duplicate candidates retain both records, explain the matched fields and require review before grouping.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R14], [R15]

ENGINEERING NOTE 038 |

Quarantine, repair and reprocess

Purpose and product decision. Data failures should be recoverable. Quarantine allows a pilot analyst to understand problems instead of losing evidence or blocking all useful rows.

Implementation contract. Store rejected row payload or safe redacted excerpt, batch ID, error codes, parser version and status. Offer corrected mapping or explicit override path, then create a new processing attempt with lineage to the original.

Failure modes and evidence constraints. Avoid retaining sensitive raw rows longer than needed. A broken import should not poison previously accepted analytics or partially replace a working dataset.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to quarantine, repair and reprocess. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Tests confirm transaction rollback, stable IDs after reprocess, redaction of errors, and a complete audit trail for manual overrides.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2]

ENGINEERING NOTE 039 |

ETL orchestration and source adapters

Purpose and product decision. Each connector should share a small interface for discovery, fetch/import, validate, normalize and report; not every source needs a scheduled poller.

Implementation contract. Define connector metadata, configuration validation, retry policy, timeout, per-source rate limits, fixture mode and deterministic transforms. Store snapshots when licensing permits. Keep the local file import interface first and implement live fetches as opt-in modules.

Failure modes and evidence constraints. A generic plugin framework can become complexity without value. Network retries can duplicate work or hide persistent schema changes.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to etl orchestration and source adapters. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A fake connector passes the same contract as a live one. Tests simulate timeout, HTTP error, unexpected schema and partial fetch, and the source status reflects the failure.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R5]

ENGINEERING NOTE 040 |

API discovery and connector policy

Purpose and product decision. A source adapter should be built only after a specific endpoint or download resource has been verified. Live API documentation may be outdated, and a public landing page is not a service guarantee.

Implementation contract. Capture request constraints, authentication requirements, query parameters, pagination, date coverage, units, timeouts, rate rules and status. Use fixed allowlisted hosts and predictable request parameters. Place all URLs in configuration rather than model-generated requests.

Failure modes and evidence constraints. Never let an agent call arbitrary URLs, submit third-party forms, or follow redirect chains to private-network addresses. External payloads are untrusted input.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to api discovery and connector policy. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A connector can run in fixture-only mode with no network. Network integration tests require explicit opt-in and prove rate and timeout limits.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R5], [R14], [R20]

ENGINEERING NOTE 041 |

Data freshness and staleness

Purpose and product decision. A stale source can make an analysis appear current when it is not. Freshness needs to be visible at the dataset and dashboard level.

Implementation contract. Define freshness expectation per source, distinguish expected refresh from observed refresh, display last source observation and last successful ingestion. For static data, show publication/vintage date rather than labelling it stale every day.

Failure modes and evidence constraints. Do not invent a universal 24-hour SLA. A manually updated official PDF may have no refresh cadence, and some satellite products have long latency.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to data freshness and staleness. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A stale-feed simulation changes UI status and report caveats. All reports mention the observation window, not just the generation time.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R2], [R5], [R8]

ENGINEERING NOTE 042 |

Baseline descriptive analytics

Purpose and product decision. The first useful analytic layer should be deterministic and explainable: counts over time, reports by category, top locations, repeat-location frequency and source coverage. These establish product value without waiting for a complex model.

Implementation contract. Define metrics using canonical records and explicit date windows. Use stable filters for place/category/status and return both aggregate values and contributing record IDs. Create summary functions that power dashboards and exports so they cannot disagree.

Failure modes and evidence constraints. A map hotspot chart and a table can tell different stories if one filters invalid coordinates and the other includes them. Each metric must show denominator and exclusions.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to baseline descriptive analytics. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. For a fixed fixture, every KPI matches a hand-calculated expected result. A reviewer can trace a bar or hotspot back to source records and understand the chosen time window.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R17], [R21]

ENGINEERING NOTE 043 |

Recurrence and hotspot definition

Purpose and product decision. A “hotspot” is a chosen statistic over a chosen spatial unit and time window, not a natural fact. Different choices—raw report count, unique report days, recurrence rate or exposure-normalized count—produce different rankings.

Implementation contract. Offer multiple views with explicit definitions: report density per grid/area, repeated reports within a distance-time window, and recurring report count by ward. Use a stable grid or boundary version. Display raw count alongside normalized values.

Failure modes and evidence constraints. Sparse data cause unstable rankings. A single cluster can dominate a tiny grid; many reports can be duplicates; high population/road length can inflate counts. Avoid calling a high-report place the “worst” area.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to recurrence and hotspot definition. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The UI exposes the metric definition, minimum sample threshold and source coverage. Test edge cases such as empty data, single record, tied values and changing grid size.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R17], [R19], [R22]

ENGINEERING NOTE 044 |

Spatial clustering options

Purpose and product decision. Spatial clustering can find concentrations but cannot establish causality or public-service failure. Start with simple grid aggregation and compare any clustering algorithm against it.

Implementation contract. Candidate methods include fixed hex/grid counts, DBSCAN for point density, and spatial joins to known boundaries. Use interpretable parameters, select distance units appropriate to the CRS and show clusters as observed-report groups, not official incidents.

Failure modes and evidence constraints. DBSCAN output changes with distance/MinPts and can label sparse points as noise. Geography and density differ by issue category; one global parameter set may not be appropriate.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to spatial clustering options. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. On labelled fixture maps, compare clusters to a simple baseline; document why parameters were chosen. A map tooltip states algorithm, parameters and sample count.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R22], [R23]

ENGINEERING NOTE 045 |

Temporal trend and seasonality

Purpose and product decision. Time series views help separate a one-week spike from long-running recurrence. Historical counts may be incomplete, with source publication and reporting practices changing over time.

Implementation contract. Aggregate into a suitable interval only when sample volume supports it. Show raw points or confidence bands alongside smoothed trend. Annotate known source gaps, import starts and data-definition changes.

Failure modes and evidence constraints. An apparent decline may mean the source stopped updating. Avoid extrapolating from a few weeks to monsoon seasonality or interpreting correlation as cause.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to temporal trend and seasonality. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Tests include irregular dates, missing periods and zero-event intervals. UI makes absence of reports distinct from verified absence of events.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R21]

ENGINEERING NOTE 046 |

Exposure-normalized comparisons

Purpose and product decision. Comparing places without considering exposure can mislead. If one area has more roads, residents or reporting access, it can accumulate more reports even without a higher underlying incident rate.

Implementation contract. Where trusted denominators exist, provide optional rates per resident, road-kilometre or area. Record denominator source, vintage and approximation. If no denominator is available, show counts only and explicitly warn that cross-area rankings are not exposure-adjusted.

Failure modes and evidence constraints. Population grids, road lengths and boundary sizes are estimates with different dates and methods. Rates from mismatched vintages may be worse than counts.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to exposure-normalized comparisons. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Every normalized metric declares numerator, denominator and source. If the denominator is missing or zero, return “not calculated” rather than zero.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R16], [R17], [R22]

ENGINEERING NOTE 047 |

Rainfall-context analysis

Purpose and product decision. Rainfall can be a useful context variable for investigations of waterlogging and drainage reports. It is not sufficient proof of causation, nor does aggregate rainfall equal local street conditions.

Implementation contract. Align rainfall observation windows with report times; explore lagged correlations and event windows only after time semantics are clear. Use station/district/grid level metadata and show the spatial scale difference between rain and issue data.

Failure modes and evidence constraints. Small datasets, confounders, missing reporting and seasonal effects can create spurious links. Do not build an operational flooding alert from correlation alone.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to rainfall-context analysis. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A notebook/report states sample size, missingness, lag choices, uncertainty and alternatives. At least one negative/control analysis tests whether apparent correlation might be a general seasonal trend.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R5], [R7], [R8], [R21]

ENGINEERING NOTE 048 |

Evidence-first briefing

Purpose and product decision. The report generator should summarize data with deterministic text and tables first. GenAI can improve readability, but core numbers, ranking and source listing must originate from trusted code paths.

Implementation contract. Use a typed report object containing selected area, date window, included sources, computed observations, notable patterns, caveats, links and reviewer status. Render the object into HTML/PDF; allow LLM rewriting only for narrative fields validated against the object.

Failure modes and evidence constraints. Models may fabricate locations, interpret data incorrectly or omit caveats to make a concise story. Do not let the model recompute metrics from prose.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to evidence-first briefing. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A report fixture has expected numeric values and cited source links. A test fails if an LLM-only phrase introduces new figures or a statement cannot be connected to a support field.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19]

ENGINEERING NOTE 049 |

SQL metric layer

Purpose and product decision. SQL provides reproducibility, filtering and aggregation, but definitions can drift if each view writes its own query. Centralize frequently used metrics as views or tested query functions.

Implementation contract. Use parameterized queries and a whitelist of dimensions and measures. Expose aggregate results through an internal analytics service. Store metric version/SQL identifier with analysis runs when reproducibility matters.

Failure modes and evidence constraints. A dashboard query that scans all raw JSON may be slow and hard to test. A model-generated query must never have write permissions.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to sql metric layer. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Expected result tables are used in unit tests. A query review checks filters, null semantics, indexes, aggregation grain and performance on representative data volume.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R20], [R23]

ENGINEERING NOTE 050 |

Dashboard information architecture

Purpose and product decision. A map alone is not enough. Analysts need a quality snapshot, trend overview, hotspot list, record explorer and source evidence. Build a hierarchy that starts with scope and limitations and then drills into specifics.

Implementation contract. Recommended pages: Overview, Map & hotspots, Time trends, Incident explorer, Source/data health, Evidence library, Analysis brief, and Settings/Import. Use consistent global filters and a visible “demo/synthetic data” badge.

Failure modes and evidence constraints. Too many KPIs obscure what matters. Avoid red/green colour as the only encoding and provide tabular alternatives for accessibility.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to dashboard information architecture. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A first-time evaluator can import a dataset, locate a recurring area, inspect raw source rows and export a brief without consulting the developer.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R16], [R17]

ENGINEERING NOTE 051 |

Data lineage in the UI

Purpose and product decision. Every chart and generated claim should have a lineage path. This is essential for credibility when sources conflict or models provide explanations.

Implementation contract. Support click-through from visualization to aggregate definition, underlying canonical record IDs and source item URL/file/page. For derived model outputs, include model version, run time, features used and evaluation state.

Failure modes and evidence constraints. Source URLs can be unavailable or changed. Preserve the source name, accessed-at date, cached metadata where lawful and content hash; do not make the live link the only provenance record.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to data lineage in the ui. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A lineage test verifies that a report statistic can be traced to input rows and source cards. Unavailable sources remain cited with an explicit retrieval failure status.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2], [R19]

ENGINEERING NOTE 052 |

Decision-support recommendation logic

Purpose and product decision. The output should propose investigation or maintenance planning steps, not command city operations. Recommendations may be prioritised by recurrence, report age, source agreement and uncertainty only if those dimensions are visible.

Implementation contract. Build an explicit rule/scorecard baseline before using a model to rank actions. Each rule has input, threshold, rationale, owner and review date. Provide “why this appears” and “what evidence could change this suggestion.”

Failure modes and evidence constraints. Do not turn arbitrary weighted scores into objective public-harm rankings. Avoid assigning blame to departments or people without verified authoritative records.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to decision-support recommendation logic. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A human reviewer can reject, edit or accept a draft and record the reason. Recommendation text includes evidence and caveats, and the default state is draft.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19]

ENGINEERING NOTE 053 |

ML scope selection and model card

Purpose and product decision. ML belongs only where it improves a measurable workflow beyond deterministic rules. The first candidate tasks are complaint category classification, duplicate candidate ranking and report-volume forecasting if sufficient labels exist.

Implementation contract. Before each model, specify target variable, unit of prediction, label provenance, training window, intended user, baseline, metric, decision threshold and failure costs. Track experiments and serialize model/data versions.

Failure modes and evidence constraints. The existence of a model in the stack does not make the product better. A small pilot may have too little labelled history for deep learning; a well-tested heuristic can be superior.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to ml scope selection and model card. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Each model feature has a model card, holdout results, error examples, limitations and an off-switch. The app can fully operate when the model package is not installed.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R21]

ENGINEERING NOTE 054 |

Complaint category classification

Purpose and product decision. Text classification can normalize different descriptions into a controlled set such as waterlogging, road damage, waste, drainage, lighting and other. Labels must be based on the text, not hidden assumptions about location or neighbourhood.

Implementation contract. Start with a small human-reviewed training set and a TF-IDF plus linear baseline; compare a lightweight local pretrained encoder only if it provides validated gains. Keep confidence and top alternatives so uncertain cases can be reviewed.

Failure modes and evidence constraints. Categories can overlap, text can be multilingual or code-mixed, and an "other" label may be underused. Do not auto-classify high-impact reports without a human review path.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to complaint category classification. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain "unknown / unavailable" rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Report macro-F1 and per-class precision/recall, confusion matrix and a reviewed error set. Add a minimum-data threshold; if too few labels exist, use keyword rules labelled as such.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R24]

ENGINEERING NOTE 055 |

Duplicate-report detection

Purpose and product decision. Duplicate detection should assist a reviewer in grouping possible reports of the same issue. It should not delete records or decide that a person's report is redundant by itself.

Implementation contract. Build candidate pairs using spatial distance, time proximity, text similarity and category. Begin with a deterministic score and inspect a sample of positive/negative pairs; only later train a pair classifier if reviewed labels accumulate.

Failure modes and evidence constraints. Repeated issues in the same location on different dates are often recurrence, not duplicates. Similar text templates can falsely merge different events. A missing location lowers certainty.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to duplicate-report detection. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Measure precision at the top-K candidates and reviewer acceptance. The UI shows matched dimensions, preserves both reports and requires an explicit merge/associate action.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R15], [R19], [R24]

ENGINEERING NOTE 056 |

Time-series forecasting of report volume

Purpose and product decision. Forecasting should start with the narrow measurable target “number of reports for a defined area/category per day or week,” not a claim to predict physical incidents. A report-volume forecast may reflect data collection behavior as much as underlying conditions.

Implementation contract. Establish naive last-period, seasonal-naive where enough history exists, moving average and linear regression baselines. Use chronological splits and backtesting windows. Forecast intervals should be shown when supported.

Failure modes and evidence constraints. Random train/test shuffling leaks future patterns. A forecast can appear accurate in quiet periods while failing during extremes. A lack of long historical data means no forecast should be surfaced.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to time-series forecasting of report volume. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Compare MAE/RMSE to a baseline on multiple future windows and show the baseline score. The UI labels output “experimental report-volume estimate” with training period and limitations.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R21]

ENGINEERING NOTE 057 |

Flooding risk: what can and cannot be predicted

Purpose and product decision. Street-level flood prediction is out of MVP scope unless there are reliable incident labels at suitable spatial/time resolution and enough history. District warnings and coarse rainfall products are not enough to truthfully claim street-level probability.

Implementation contract. A future research path could combine rainfall intensity/accumulation, elevation/drainage, historic validated flood observations and drainage assets. Separate nowcast, susceptibility mapping and observed waterlogging; they are different tasks requiring different validation.

Failure modes and evidence constraints. A model could systematically understate risk in areas with low reporting coverage. False reassurance and false alarms have real consequences. Never use this tool for emergency decisions.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to flooding risk: what can and cannot be predicted. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The MVP reports observed issue recurrence and weather context separately. Any risk model needs documented labels, calibration, recall at relevant thresholds, subgroup checks and approval before release.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R5], [R6], [R7], [R8], [R19]

ENGINEERING NOTE 058 |

Anomaly detection and explainable flags

Purpose and product decision. Anomaly detection can flag a sudden change in reporting volume, unusual source behavior or implausible values. It is not automatically a physical incident detector.

Implementation contract. Start with robust rules such as deviation from a trailing median when history is adequate. Add statistical methods only after the expected distribution and seasonal pattern are inspected. Flag reasons, comparison window and input quality.

Failure modes and evidence constraints. A new import, source schema change, publicity event or data outage can create an apparent anomaly. Model drift and sparse history make thresholds unstable.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to anomaly detection and explainable flags. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A seeded fixture produces expected anomaly flags and explanatory values. Real outputs state whether the flag relates to data quality or observed reports, and include a false-positive review workflow.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R21]

ENGINEERING NOTE 059 |

Deep learning decision gate

Purpose and product decision. Deep learning should be an experiment rather than a mandatory component. It is justified only if data scale, representation or image/text task warrants it and the result beats an appropriate baseline.

Implementation contract. For text, evaluate compact pretrained encoders; for image uploads, consider transfer learning only after a lawful, diverse labelled image dataset exists. Track hardware, memory, batch size, seed, dependency version and model file size.

Failure modes and evidence constraints. On a MacBook Air M1, memory and inference speed are finite. Large foundation models may swap heavily. Downloaded weights have licences and possible usage restrictions separate from code licence.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to deep learning decision gate. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A written experiment compares latency, memory, quality and complexity against a simpler baseline. If it does not win, document the no-go decision; this is a valid engineering result.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R24]

ENGINEERING NOTE 060 |

Computer vision as optional extension

Purpose and product decision. Photo-based road damage or waste classification may be valuable later, but only after a photo dataset, consent/licence review and label protocol exist. A pretrained demo on unrelated images proves only that inference executes.

Implementation contract. The data card must describe image source, location/time metadata, labels, annotator process, class balance, duplicates and geographic diversity. Blur faces and plates if images are used; remove EXIF metadata when location detail is unnecessary.

Failure modes and evidence constraints. A visual model can be fooled by lighting, camera angle, rain, occlusion or local surface differences. Detection output cannot establish severity or ownership of an issue.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to computer vision as optional extension. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Do not enable the feature in the MVP until precision/recall per class, out-of-distribution limits, privacy handling and reviewer workflow are documented.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R10]

ENGINEERING NOTE 061 |

Feature engineering and leakage prevention

Purpose and product decision. Predictive features should only use information available at prediction time. A late-updated status, post-event official report or later-reported complaint can leak the target.

Implementation contract. Write a feature manifest with source, availability time, window, transformation and null handling. Generate training examples as-of a timestamp. Fit scalers and encoders on training folds only. Record all feature code versions.

Failure modes and evidence constraints. Using future data creates excellent metrics that collapse in real use. Spatial leakage can also occur if nearly identical neighbourhoods appear in train/test and the claim is geographic generalisation.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to feature engineering and leakage prevention. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Automated tests assert each feature timestamp is not after prediction time. Backtests document temporal split and, when relevant, spatial holdouts.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R21]

ENGINEERING NOTE 062 |

ML evaluation and calibration

Purpose and product decision. Model metrics should match the decision. Class imbalance calls for macro metrics and per-class errors; a forecast needs temporal backtests; probability claims need calibration.

Implementation contract. Use untouched test windows, compare simple baselines, preserve confusion examples and evaluate at operationally plausible thresholds. Report confidence intervals or variability across windows when sample size permits.

Failure modes and evidence constraints. One aggregate metric can conceal failure on low-volume categories or underserved areas. Avoid tuning on the test set. Small datasets cannot support strong claims even if the score looks high.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to ml evaluation and calibration. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. No model merges or auto-action can occur until the acceptance threshold is agreed and evidence is recorded. The evaluation script is reproducible from pinned data and dependency versions.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R21]

ENGINEERING NOTE 063 |

Fairness and coverage bias

Purpose and product decision. Complaint records are a proxy for observed reporting, not a neutral sample of civic harm. Lower digital access, language barriers, fear or lack of awareness can suppress reporting in some areas.

Implementation contract. Where safely available, compare coverage across areas and categories; keep the absence of denominators explicit. Provide manual upload routes and multiple ways to inspect data. Avoid inferring demographic traits about individuals.

Failure modes and evidence constraints. Ranking areas by report volume can reinforce over-surveillance or underinvestment depending on interpretation. Do not use personally identifying data or protected attributes as shortcuts.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to fairness and coverage bias. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Include a “coverage and representation” panel in the analysis brief. Reviewers see what areas and dates are missing before interpreting rank.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R16], [R18], [R19]

ENGINEERING NOTE 064 |

NLP pipeline and normalization

Purpose and product decision. NLP begins with text cleaning that preserves meaning and source. Do not erase local terms, landmark names, languages, negation or numeric details as part of blanket normalization.

Implementation contract. Store raw text under access limits; derive cleaned text, language guess, normalized tokens and extracted entities separately. Make each transformation versioned and reversible from raw source where lawful. Use regex/rules for robust simple tasks before a large model.

Failure modes and evidence constraints. Text may contain addresses, names, phone numbers, insults or instructions aimed at an AI. Treat all imported content as untrusted and never interpret embedded instructions as system policy.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to nlp pipeline and normalization. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Unit tests use representative English, Hindi, Marathi and code-mixed examples if they occur in the pilot. The user can inspect raw vs normalized text with sensitive fields masked.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R24]

ENGINEERING NOTE 065 |

Multilingual and code-mixed civic text

Purpose and product decision. Indian civic reports may combine English with regional languages, transliteration, abbreviations and colloquial landmarks. A model that works on English benchmark text may fail on real local phrasing.

Implementation contract. Start by inventorying actual languages from permitted data and define category labels with bilingual examples. Evaluate transliteration and code-switching samples separately. Choose local models only after comparison and licence check.

Failure modes and evidence constraints. Do not assume all local-language words can be translated without context. A low-confidence language or category should route to human review.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to multilingual and code-mixed civic text. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Report accuracy by language/phenomenon when the sample size supports it. If not enough examples exist, state the gap instead of aggregating away failures.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R24]

ENGINEERING NOTE 066 |

Named entity and place extraction

Purpose and product decision. Entity extraction can help identify roads, landmarks, drains and neighbourhood names from free text, but extracted entities are candidates, not verified coordinates or administrative facts.

Implementation contract. Use a gazetteer built from approved local place names and aliases, plus regexes for codes. Return span, normalized candidate, extraction method, confidence and unresolved state. Keep the full sentence as evidence.

Failure modes and evidence constraints. Names may refer to people or businesses rather than locations; the same landmark can exist in multiple areas. A model can hallucinate an entity not present in the source.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to named entity and place extraction. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Each extracted place can be clicked to inspect source text. Measure precision/recall on a small annotated sample before using it to move points on the map.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R11], [R22], [R24]

ENGINEERING NOTE 067 |

Text embeddings and semantic similarity

Purpose and product decision. Embeddings can retrieve semantically related reports when exact words differ. Similarity is a candidate-generation signal, not an identity decision or truth score.

Implementation contract. Compare a lightweight local sentence-transformer to TF-IDF/cosine on a small manually labelled set. Cache embeddings by text hash and model/version. Store vectors locally; avoid a hosted embedding API to meet the zero-paid-call rule.

Failure modes and evidence constraints. Multilingual quality, small-text noise and local place terms can degrade semantic similarity. Model weights have independent licences and can be too large for the laptop.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to text embeddings and semantic similarity. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Measure retrieval precision at K and latency on the intended hardware. Keep an exact-match/rule baseline and a fully functional mode without the embedding model.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R24]

ENGINEERING NOTE 068 |

NLP privacy redaction

Purpose and product decision. Before indexing documents or prompting a local LLM, identify whether personal data is necessary. The lowest-risk path is to remove or mask names, contact details and precise residential addresses from the retrieval text when not essential.

Implementation contract. Create a redaction stage with patterns for Indian phone numbers, email, obvious IDs and configurable sensitive tokens; treat it as best-effort, not perfect anonymization. Keep a secure mapping only if there is a justified workflow and retention plan.

Failure modes and evidence constraints. Regex can miss or over-redact. Reversible pseudonyms are still sensitive data. Local execution lowers external exposure but does not eliminate device and access risks.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to nlp privacy redaction. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Privacy tests include common patterns, obfuscated phone numbers, false positives and export checks. Raw/private fields are never emitted into logs or LLM prompt traces by default.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19]

ENGINEERING NOTE 069 |

RAG use case and evidence policy

Purpose and product decision. RAG should answer questions about source documents, municipal guidance and historical incident summaries with citations. It must not turn guidance into a claim that a particular drain, street or current situation has been verified.

Implementation contract. Every indexed chunk carries title, publisher, source URL/file, page/section, publication date, downloaded date, licence, version/hash and sensitivity. Retrieval returns both text and provenance; answer templates demand citations and an uncertainty response when evidence is insufficient.

Failure modes and evidence constraints. A retrieved passage can be outdated, irrelevant or malicious. Similarity score alone does not prove authority. Use policy documents as context, not executable instructions.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to rag use case and evidence policy. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A source-grounded benchmark contains answerable and unanswerable questions. Answers are checked for support, correct citation and appropriate abstention.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R6], [R7], [R16], [R24]

ENGINEERING NOTE 070 |

Document ingestion for knowledge base

Purpose and product decision. Knowledge-base ingestion must preserve paragraph/page boundaries and source metadata so an analyst can verify the answer. PDFs can contain text, tables, scanned images and external links.

Implementation contract. Build a staged pipeline: source registration, file hash, text extraction, page/section segmentation, privacy scan, chunk generation, embedding, index version and evaluation report. Keep extracted text separate from the original document.

Failure modes and evidence constraints. Chunking can break the qualifiers from their claims. OCR can introduce incorrect names or numbers. Prompt injection in documents should be treated as untrusted content.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to document ingestion for knowledge base. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Every returned answer can point to an exact document/page or section. The index can be rebuilt deterministically from the same source manifest.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R24]

ENGINEERING NOTE 071 |

Chunking and retrieval strategy

Purpose and product decision. Choose chunking for answer evidence, not a magic token count. Keep section headings and table context attached where meaningful. Use overlapping windows only where they improve retrieval.

Implementation contract. Begin with fixed-size paragraph-aware chunks and metadata filters by source type/date/area. Compare keyword BM25-style retrieval to local embedding retrieval and, if justified, hybrid ranking. Pin tokenizer and embedding model versions.

Failure modes and evidence constraints. Huge chunks bury relevant passages; tiny chunks lose context; embedding retrieval misses exact codes and proper nouns. A vector database adds operational cost if a local flat index suffices.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to chunking and retrieval strategy. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Benchmark recall at K on known source passages and exact-code questions. Each result displays relevance evidence and provenance; stale/withdrawn documents can be removed from the current index while retained in the audit record.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R24]

ENGINEERING NOTE 072 |

RAG answer generation and citations

Purpose and product decision. A RAG answer should separate retrieved facts, deterministic database results and the model's interpretation. Citations must reference actual retrieved chunks, not be generated as guessed URLs.

Implementation contract. Use a structured answer schema: answer, claims[], citations[], uncertainty, missing_data, and suggested_follow_up. Each claim needs one or more source chunk IDs. Validate citations against retrieved results and allow an abstain state.

Failure modes and evidence constraints. Models can cite a source that does not support the claim or synthesize a claim from multiple weak passages. Require human verification for policy-sensitive or consequential recommendations.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to rag answer generation and citations. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A citation validator rejects nonexistent chunk IDs and empty source metadata. An evaluation set includes conflicting sources, outdated policy, no-answer questions and malicious instructions in retrieved text.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19], [R24]

ENGINEERING NOTE 073 |

RAG evaluation framework

Purpose and product decision. RAG evaluation must measure retrieval and generation separately. A plausible response is insufficient if the retrieved context does not actually support it.

Implementation contract. Build a small manually verified set of questions, expected supporting passages, answerable/unanswerable label and required caveats. Track retrieval recall@k, citation precision, answer factual support, abstention behavior, latency and memory.

Failure modes and evidence constraints. LLM-as-judge may be used as a secondary signal but is not ground truth. Human-authored answer keys and sampled reviews are needed; evaluation examples must not all come from the same document.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to rag evaluation framework. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A regression report must be generated after any chunker, embedding model, prompt or index change. No model switch is accepted based on anecdotal demos alone.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R24]

ENGINEERING NOTE 074 |

Prompt injection and retrieved document safety

Purpose and product decision. Any public report, uploaded PDF or retrieved document can contain instructions targeting the model. The model must treat retrieved text as evidence, not policy or tool authorization.

Implementation contract. System policy lives outside the retrieved corpus. Delimit retrieved passages, instruct the model to ignore embedded instructions, restrict tool permissions independent of prompts, and validate output schemas. Do not place secrets in context.

Failure modes and evidence constraints. Prompt wording alone is not a security boundary. An agent with powerful tools remains unsafe even if it is told not to misuse them.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to prompt injection and retrieved document safety. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Create adversarial fixtures that request data exfiltration, policy override, arbitrary URL access and SQL writes. Tests verify that tool authorization blocks actions even if the model complies with malicious content.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 075 |

RAG source conflict and freshness

Purpose and product decision. Civic guidance may have superseded versions, and sources may disagree. The UI should preserve source date and distinguish current official publication from secondary commentary.

Implementation contract. Track publication/effective/withdrawal dates when available. Define a precedence rule only when authority and date support it; otherwise present conflicts side by side. Provide a “review required” state for unresolved policy contradictions.

Failure modes and evidence constraints. The newest page is not always more authoritative; an unofficial blog can be newer than a government order. Do not choose the answer based solely on retrieval score or crawl date.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to rag source conflict and freshness. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Conflict tests return both evidence sources, explain the conflict and abstain from declaring a definitive resolution without an explicit rule.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R2], [R7], [R18]

ENGINEERING NOTE 076 |

Local LLM and zero paid API policy

Purpose and product decision. The zero-paid-API principle means inference, embeddings and core analytics run locally or are deterministic. Network downloads of model weights may be required once; those downloads are not inference APIs but still require licence, storage and bandwidth review.

Implementation contract. Design model provider interfaces with a deterministic stub and optional local adapter. Select a compact model based on task quality and M1 RAM. The core application and tests must run without Ollama or a downloaded model.

Failure modes and evidence constraints. Model licences differ from application code licences. Large local models can exhaust RAM, swap heavily or produce slow responses; the developer must not be forced into a paid fallback.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to local llm and zero paid api policy. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A clean install can exercise the core app in no-model mode. Local mode exposes model ID/version and fails with an actionable error if missing; it never silently calls a remote API.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R24]

ENGINEERING NOTE 077 |

Agent role and constrained autonomy

Purpose and product decision. The agent's job is to orchestrate allowed analysis steps and draft a reviewable brief, not to act as a free-roaming administrator. Use a bounded plan-execute-summarize loop with typed tool outputs.

Implementation contract. The permitted tools may include search approved documents, query whitelisted analytics, list source health, compare time windows, retrieve related records and create a draft report. Each tool has schema validation, timeout, row limits, explicit permissions and audit logging.

Failure modes and evidence constraints. An agent can loop forever, call tools repeatedly, over-query sensitive rows or treat untrusted text as instructions. The LLM must not decide its own permissions or add new tools.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to agent role and constrained autonomy. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Set maximum steps, elapsed time, context tokens and result size. A tool call is rejected unless it matches a registered schema; stop/timeout behavior is tested.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19], [R20]

ENGINEERING NOTE 078 |

Agent planning and state machine

Purpose and product decision. A simple state machine is easier to debug than a swarm of agents. Make the workflow explicit: accept user question, classify intent, validate scope, plan allowed tools, execute bounded calls, build evidence bundle, draft answer and return for review.

Implementation contract. Persist run ID, user goal, approved area/date filters, each tool name and arguments, output hashes, errors and final citations. Keep sensitive tool output redacted. Allow cancellation and deterministic replay using stored tool results where permitted.

Failure modes and evidence constraints. Do not let model-generated plans broaden geography, time range or permission beyond user-approved scope. A plan is not authorization.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to agent planning and state machine. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Tests cover invalid intent, no-data response, failed tool, cancellation, limit exceeded and replay. The system always returns either a supported answer or a clear abstention.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 079 |

Read-only SQL tools for the agent

Purpose and product decision. The model should not execute arbitrary SQL text against the database. Natural-language questions can map to a finite registry of named analytical tools whose parameters are validated.

Implementation contract. Expose functions such as `count_reports`, `trend_by_week`, `top_reported_areas`, `related_observations` and `source_health`. Each function uses parameterized SQL or a query builder, a read-only connection, limit, timeout and approved columns.

Failure modes and evidence constraints. Even SELECT statements can expose private data or trigger expensive joins. SQL comments, multi-statements, unsafe functions and parameter tricks should never be routed from raw model text to the DB engine.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to read-only sql tools for the agent. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Security tests prove the agent cannot modify tables, access disallowed fields, exceed row limits or pass unchecked SQL through the interface.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R20]

ENGINEERING NOTE 080 |

Human review and approval model

Purpose and product decision. CityPulse outputs are proposals. Human review is required before publishing a brief, creating any durable recommendation state or using the information outside the local tool. The MVP should omit municipal submission and outbound contact actions.

Implementation contract. A review screen shows evidence, caveats, generated draft, reviewer edits, disposition and timestamp. Preserve the original machine draft and a diff for audit. A rejection is an outcome, not an error.

Failure modes and evidence constraints. Approval should not be a cosmetic button that triggers a hidden high-impact action. Keep all external side effects disabled by design unless future owners explicitly approve a threat and legal review.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to human review and approval model. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Tests verify no external call occurs when a recommendation is drafted or approved. The audit record says who reviewed what, and exports show review status clearly.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19]

ENGINEERING NOTE 081 |

Agent tool authorization

Purpose and product decision. Tool access must be enforced in code independently of the model prompt. A tool catalog and policy layer should allow or deny each operation by role, scope and data sensitivity.

Implementation contract. Define tools as typed functions with input validation, allowlisted parameters, output schemas, timeouts and resource budgets. Keep policy evaluation in the application layer and use capability-based access where appropriate.

Failure modes and evidence constraints. If the model can construct arbitrary URLs, paths, shell commands or SQL, it can cross the intended boundary. Never give the agent a general terminal tool in the deployed app.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to agent tool authorization. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A policy matrix tests every tool against allowed, denied, malformed and out-of-scope requests. Denied calls are logged without leaking their hidden data.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 082 |

Agent observability and replay

Purpose and product decision. Debugging an agent requires a safe record of what it attempted and why. Logs should explain tool sequence and failure without retaining entire sensitive prompt contents by default.

Implementation contract. Record correlation ID, model name/version, prompt/template version, tool schema version, durations, token estimates, citations and errors. Use redaction, retention limits and an exportable trace for synthetic scenarios.

Failure modes and evidence constraints. Full prompt logging may collect addresses, names or sensitive text. LLM outputs and tool returns may be too large to store indefinitely.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to agent observability and replay. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A synthetic run can be replayed with deterministic tool outputs and produces the same evidence bundle. A privacy audit confirms logs omit sensitive fields by default.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 083 |

API boundary and error design

Purpose and product decision. FastAPI or another backend framework may be appropriate, but the stack is a decision for repository reconnaissance. The API should keep parsing, domain logic, models, persistence and transport concerns separate.

Implementation contract. Design stable API contracts for health, source status, import preview/import, list/search observations, aggregate metrics, geo features, knowledge search, report drafts and review events. Use typed request/response schemas and versioning discipline.

Failure modes and evidence constraints. Do not expose raw database exceptions, file system paths or stack traces to users. Avoid endpoints that accept arbitrary file paths or unauthenticated administrative actions.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to api boundary and error design. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Contract tests validate schemas, pagination, error codes, boundary values and permission checks. The API can be run without model weights and external APIs.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R20]

ENGINEERING NOTE 084 |

Authentication, roles and local-first assumption

Purpose and product decision. Single-user local mode is a good initial scope, but it must be described honestly rather than treated as secure multi-user SaaS. Binding a local server to the network changes the threat model.

Implementation contract. Default to loopback-only serving; document when a development server binds all interfaces. If adding users later, define roles for viewer, analyst and reviewer, server-side checks and audit. Do not rely on UI hiding alone.

Failure modes and evidence constraints. Local applications can still leak through shared computers, browser storage, debug endpoints or file permissions. If deployed for a group, revisit authentication, secrets, backup and tenancy before exposing any personal data.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to authentication, roles and local-first assumption. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A test confirms protected routes reject missing/invalid credentials when multi-user mode exists, and the deployment checklist flags non-loopback binding.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 085 |

Upload security and parser hardening

Purpose and product decision. File uploads are a major attack surface even in a student project. Treat CSV, GeoJSON, PDF and image files as hostile input and limit the resources parsers can consume.

Implementation contract. Set maximum size, row count, nested archive depth, execution time and output count. Do not execute macros or formulas. Use temporary directories with safe names and cleanup; validate extension and actual content rather than extension alone.

Failure modes and evidence constraints. A huge file or maliciously crafted geometry can hang the app. Filenames can contain path traversal. Formula injection can harm spreadsheet users when a CSV export is opened.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to upload security and parser hardening. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Test zip bombs/oversized files using harmless synthetic fixtures, path traversal filenames, malformed encodings and formula-leading cells. Fail closed with friendly errors and no partial canonical mutation.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R20]

ENGINEERING NOTE 086 |

Privacy and personal data

Purpose and product decision. Civic issue reports may contain personal information even if their topic is public. The MVP should minimise collection, minimise retention and make sensitive data absent from model indexes where possible.

Implementation contract. Classify fields by sensitivity, purpose and retention. Use redacted analytical views, no contact data in embeddings, local storage by default and a documented deletion/export process. Review India's applicable data-protection obligations and obtain qualified advice before real public launch.

Failure modes and evidence constraints. Do not imply local-only processing automatically makes all processing lawful. Dataset licence does not grant the right to republish personal details. Re-identification can occur from precise location and time combinations.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to privacy and personal data. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Create a data inventory, privacy threat register and deletion test. Use synthetic data for demos and remove sensitive values from logs, traces, screenshots and sample repositories.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19]

ENGINEERING NOTE 087 |

Security threat model

Purpose and product decision. Security should cover the whole path: source files, parsing, SQL, models, prompts, retrieval, exports, map services and the user's machine. Public-sector data products can cause harm through inaccurate outputs even without a classic exploit.

Implementation contract. Use threat modelling with assets, trust boundaries, attacker capability, impact, likelihood, mitigations, residual risk and owner. Cover injection, broken access control, data exfiltration, malicious document instructions, dependency compromise and false claims.

Failure modes and evidence constraints. A threat list is not a substitute for tests. A “human in the loop” does not neutralize a system that hides evidence or makes approval unavoidable under time pressure.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to security threat model. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Every high severity risk has a mitigation test or a documented release blocker. Run dependency scanning if available and keep logs of review date and scope.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19], [R20]

ENGINEERING NOTE 088 |

API security checklist

Purpose and product decision. OWASP API Security Top 10 highlights risks such as broken object-level authorization, broken authentication, unrestricted resource consumption, SSRF, misconfiguration and unsafe API consumption. Map applicable risks to actual CityPulse endpoints instead of claiming checklist completion without evidence.

Implementation contract. For each route, verify authentication/authorization assumptions, tenant/owner check if relevant, input validation, payload limit, rate limit, timeout, error sanitisation, log redaction and external host allowlist. Use dependency injection for permission checks rather than duplicating them ad hoc.

Failure modes and evidence constraints. A small local tool may still face denial of service or path traversal. SSRF can arise if a source URL is user controlled. Never fetch arbitrary URLs from uploaded records without an allowlist.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to api security checklist. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Create a route-by-route security matrix and automated tests for each relevant control. Record exceptions and why they are acceptable for local-only mode.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R20]

ENGINEERING NOTE 089 |

Open-source and model licence management

Purpose and product decision. Zero-cost does not mean unrestricted. Code packages, data, maps, model weights, fonts, sample photos and datasets may have different licences and attribution conditions.

Implementation contract. Maintain a third-party inventory with package/model/dataset, version, author, licence, origin URL, attribution, use restrictions and commercial-use status. Separate transitive package licence review from model-weights review.

Failure modes and evidence constraints. A model downloaded from a public repository may have non-commercial or attribution restrictions. OSM's ODbL can impose obligations on derivative databases. Don't publish raw data unless permitted.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to open-source and model licence management. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Release checklist blocks deployment until required notices and attributions are included. Store source URLs in the repo documentation and ensure demo data is synthetic or licence-cleared.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R2], [R3], [R11], [R9]

ENGINEERING NOTE 090 |

Environment and dependency reproducibility

Purpose and product decision. A reproducible project should install from a clean environment without relying on global packages or manual shell state. Keep dependency versions and optional heavy dependencies separate.

Implementation contract. Select package manager after repository audit. Add lockfiles, platform notes for Apple Silicon, a minimal core requirements set and optional ML/RAG extra. Document supported Python version, database setup and seed data.

Failure modes and evidence constraints. Unpinned transitive dependencies may break a future install. Some geospatial packages have native wheels/build constraints; large ML packages can use significant disk.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to environment and dependency reproducibility. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Test a clean virtual environment from a fresh clone. CI runs core tests without downloaded models or network access, and a documented optional command validates model-enabled mode.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R22], [R23], [R24]

ENGINEERING NOTE 091 |

Configuration and secrets

Purpose and product decision. Configuration should be explicit and validated. The app should not need a paid key and must not contain a hidden fallback to hosted AI services.

Implementation contract. Use environment variables or a typed local config file for paths, enabled connectors, model backend, file limits and logging. Keep example config free of secrets and provide a startup report with model/network mode.

Failure modes and evidence constraints. Secrets can leak through exception messages, notebooks, Git history or generated traces. A source URL or API token must not be embedded in public demo files.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to configuration and secrets. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Automated scan and manual review find no credentials in source files. Running with no API-related environment variables still launches core functionality.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 092 |

Local storage, backups and migration

Purpose and product decision. Local-first is cost-effective but data still needs persistence and recovery. Pick a storage strategy based on repository and data volume, not trend.

Implementation contract. SQLite may be ideal for a single-user starter; PostgreSQL/PostGIS may be justified for concurrent access and heavy spatial SQL. Migrations, export, backup and restore contracts should be explicit either way.

Failure modes and evidence constraints. Database choice affects spatial operations, deployment, packaging and test complexity. Do not add PostGIS to a small demo unless its features materially improve the MVP.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to local storage, backups and migration. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A backup can be restored into a fresh environment and reproduces the same counts. Migration tests run from an empty database and the immediately previous version.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R23]

ENGINEERING NOTE 093 |

Model/provider abstraction

Purpose and product decision. Analytics code should not depend on one LLM framework. Use a narrow application interface for classify, embed, retrieve and generate; keep deterministic implementations available.

Implementation contract. Define typed model request/response contracts, error types, cancellation, timeout, model identity and local/remote policy. The zero-cost policy should make any remote option disabled and explicit by default.

Failure modes and evidence constraints. A generic abstraction can become overengineering if it wraps every vendor concept. A local provider may fail for memory reasons and should not corrupt the app state.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to model/provider abstraction. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Tests run against a fake deterministic provider, missing provider, timeout and malformed provider output. No code path silently falls back to a paid endpoint.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R24]

ENGINEERING NOTE 094 |

Observability and application logs

Purpose and product decision. Logs should support debugging and evaluation but must not become a secondary copy of sensitive data. Use structured, bounded event logs.

Implementation contract. Track import batch, analysis run, model run and agent run IDs; record stage, duration, status and error code. Keep human-readable log levels and redact configurable patterns. Provide diagnostics export using synthetic or redacted data.

Failure modes and evidence constraints. Log volume can grow unbounded, and model prompts may include personal details. Stack traces can expose file paths or dependency state.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to observability and application logs. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Log tests assert sensitive fields are absent. The application reports source and pipeline failures with correlation IDs that connect UI error messages to local diagnostic records.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 095 |

Performance budgets for M1 laptop

Purpose and product decision. The application should feel responsive on a MacBook Air M1 with limited memory, especially in no-model mode. Large data and model tasks must be bounded and cancellable.

Implementation contract. Define realistic fixture sizes and budget p95 page latency, import duration, memory footprint and optional inference latency. Stream or chunk files, cache derived aggregates and keep expensive tasks off the UI thread.

Failure modes and evidence constraints. Do not assume GPU acceleration or enough unified memory for large LLMs. Native geospatial dependencies may behave differently by architecture.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to performance budgets for m1 laptop. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Record a baseline timing report on the target device or a representative environment. Slow tasks show progress/cancellation and do not freeze normal navigation.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R22], [R24]

ENGINEERING NOTE 096 |

Zero paid API cost architecture

Purpose and product decision. Core features should work offline after optional datasets/models are downloaded. The architecture must not require OpenAI/Anthropic or another commercial inference endpoint for operation; using Claude Code Pro for development is separate from runtime API dependencies.

Implementation contract. Implement local analytics, local database, local embedding/LLM optional adapters, downloaded public data fixtures and local map layers. For each external connector, provide a data-free fixture path and an explicit toggle.

Failure modes and evidence constraints. Public web services may have costs or policies even without per-call billing. Public basemaps can be unavailable; dataset usage can require account registration; local models consume CPU, RAM and storage.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to zero paid api cost architecture. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. An automated “offline smoke test” starts the app with network disabled or mocked and confirms import, analytics, retrieval fixture and report export work.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R12], [R13], [R24]

ENGINEERING NOTE 097 |

Deployment scenarios and decision gate

Purpose and product decision. A local demo may run on the laptop; a pilot may need a managed host, authentication and backup. Do not design production infra before a partner and data access justify it.

Implementation contract. Compare local app, single small server and container deployment based on privacy, operational burden, data size and user count. Do not require Docker if a simple venv install satisfies the repo. Deployment technology remains Claude Code's architectural choice after reconnaissance.

Failure modes and evidence constraints. Free-tier hosting terms change and may sleep, limit CPU or require a card. Data that includes personal details should not be uploaded casually to a public demo.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to deployment scenarios and decision gate. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The runbook distinguishes local demo, private pilot and public production. Each has its own threat model, secrets policy, backup, monitoring and budget estimate.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R20]

ENGINEERING NOTE 098 |

Testing strategy and pyramid

Purpose and product decision. Testing should exist at several layers: unit tests for deterministic logic, contract tests for source adapters/API responses, integration tests for the database, end-to-end tests for the core journey and adversarial tests for AI safety. The no-model core path should be testable offline.

Implementation contract. Write high-value tests with stable fixtures, not brittle snapshots of model wording. Use property-based tests for parsers/geometries if practical. Mock network requests and add a tiny curated benchmark for classification and retrieval.

Failure modes and evidence constraints. A green test suite can still miss wrong assumptions; combine tests with manual evidence review and usability testing. Tests must not call real APIs or incur paid charges by default.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to testing strategy and pyramid. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. CI runs lint/type checks, unit/integration tests and dependency checks as appropriate. The developer can run the same commands locally, and failures explain which acceptance criterion is broken.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20], [R21]

ENGINEERING NOTE 099 |

Synthetic fixtures and safe demo mode

Purpose and product decision. Synthetic data make the project demonstrable before a municipal data agreement exists. They must be clearly labelled so a user cannot mistake them for real civic records.

Implementation contract. Create seeded fixtures with realistic but fictitious IDs, timestamps, categories, coordinates near a configurable pilot region, duplicates, missing values, source delays and data quality defects. Keep all synthetic provenance fields set and document generation rules.

Failure modes and evidence constraints. Do not mix synthetic and real records in one aggregate without an explicit mode indicator. Synthetic distributions can encode assumptions that should not be presented as facts about Mumbai.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to synthetic fixtures and safe demo mode. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A screenshot/export visibly says “synthetic demo data.” A test ensures switching to real mode disables fixture injection and analytics filters show source type.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R6]

ENGINEERING NOTE 100 |

CI, release and code quality

Purpose and product decision. A credible portfolio repo should make it easy to reproduce the app and evaluate changes. CI is a guardrail for regressions rather than a substitute for design review.

Implementation contract. After repo audit, adopt the existing CI if suitable or add a small workflow for lint, tests, type checking, migration validation and packaging. Pin actions/dependencies to safe versions and avoid secrets in public jobs.

Failure modes and evidence constraints. Heavy model integration tests may be unsuitable for every CI run. Mark them separately, but ensure the mandatory suite tests the local core path and safety controls.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to ci, release and code quality. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A clean pull-request-like change set has test output, concise change log, known limitations and no committed data secrets. The user performs Git commits/pushes manually.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 101 |

Accessibility and inclusive design

Purpose and product decision. Civic insight tools must be usable beyond map experts. Maps should have equivalent tables, charts need text summaries, controls need keyboard support and colours cannot carry meaning alone.

Implementation contract. Provide clear contrast, labels, focus order, keyboard navigation, accessible validation errors and plain-language definitions of metrics. Local-language support should be added based on user needs and evaluation, not machine-translated claims.

Failure modes and evidence constraints. A dense map may exclude screen-reader users; overly technical confidence labels can confuse non-experts. Avoid red/green as the only alert indicator.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to accessibility and inclusive design. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Run a manual keyboard-only pass and automated accessibility checks where available. At least one main workflow is usable without interpreting the map visually.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R16], [R17]

ENGINEERING NOTE 102 |

Pilot design and user research

Purpose and product decision. The pilot should test whether evidence synthesis is actually useful for a real reviewer. Choose a small scope with data that can be legally handled, and establish a baseline workflow before claiming impact.

Implementation contract. Recruit one or two willing analysts/NGO/residents association participants if possible. Use a defined historical period, reviewed sample and fixed task. Ask them to verify source citations and action brief clarity.

Failure modes and evidence constraints. A student demonstration is not proof that a municipal authority will purchase or deploy the product. Do not use authority logos or imply endorsement without permission.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to pilot design and user research. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Record task time, correction rate, trust concerns, missing data and desired workflow changes. Summarize validation transparently, including negative results.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R16], [R17], [R18]

ENGINEERING NOTE 103 |

Startup discovery and differentiation test

Purpose and product decision. Commercial viability depends on a stakeholder with an urgent job and budget or institutional mandate. The product may be a research tool, consultancy accelerator or civic-tech deployment before it is a standalone SaaS.

Implementation contract. Interview users about current pain, data access, procurement, integration constraints, liability and who signs off on recommendations. Test a clickable prototype and a sample report before building advanced AI.

Failure modes and evidence constraints. A feature-rich product can lack an owner willing to pay. Municipal sales cycles, public procurement and data-sharing agreements may be long; plan a non-commercial portfolio path too.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to startup discovery and differentiation test. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Write a problem-solution validation memo with evidence, objections, current alternatives, potential buyer, expected value and next experiment. No revenue projections should be presented as fact.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R15], [R16], [R18]

ENGINEERING NOTE 104 |

Portfolio narrative and honest claims

Purpose and product decision. The repository and demo should tell a coherent story to hiring teams: a messy real-world data problem, engineering decisions, measurable evaluation, safety trade-offs and a useful interface. The story should not pretend a synthetic demo is an operational city platform.

Implementation contract. Publish an architecture diagram, data lineage diagram, model cards, evaluation reports, source register, screenshots, reproducible commands and “what did not work” notes. Include a small demo recording using synthetic or permission-cleared data.

Failure modes and evidence constraints. Avoid unsubstantiated statements like “predicts urban floods accurately” or “used by the government.” Be explicit about which results are offline experiments and which are validated with real users.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to portfolio narrative and honest claims. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A reviewer can clone the repo, run the demo, reproduce at least one metric and understand the known limitations without personal coaching.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19]

ENGINEERING NOTE 105 |

Phase 0: strictly read-only repository audit

Purpose and product decision. Claude Code must start by discovering the real state of the repository. It must not assume the stack, app structure, data pipeline or previous ProblemSpec/DataPilot work exists. This phase is reconnaissance only.

Implementation contract. Inspect top-level files, Git status and branch, manifests/lockfiles, README/CLAUDE.md, app entrypoints, tests, schemas/migrations, data contracts, model/RAG/agent code and docs. Use read-only commands; do not install dependencies, generate files, format code or alter the environment.

Failure modes and evidence constraints. Even exploratory agents can accidentally write files or run destructive shell commands. If repository context is unavailable, report that blocker rather than guessing or substituting another repository.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 0: strictly read-only repository audit. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The report separates verified facts from hypotheses and lists working, partial, missing and unknown components. It contains test commands/results when safe, risks, tech-stack options and a proposed plan, then stops for user approval.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 106 |

Phase 1: architecture and stack decision

Purpose and product decision. Only after the user approves the audit should Claude Code recommend an architecture. The decision should emerge from the actual repository, laptop constraints, zero paid runtime APIs and MVP scope.

Implementation contract. Compare at least two viable options for each major choice: app UI, API framework, relational store/spatial extension, vector retrieval, local model runtime and packaging. Evaluate continuity with existing code, local M1 support, licence, complexity, maintenance, testability and costs.

Failure modes and evidence constraints. Do not choose a fashionable tool just to make the stack look impressive. A local SQL search or simple vector index may be enough; an orchestrator library is optional if a short state machine is clearer.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 1: architecture and stack decision. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Deliver an architecture decision record with rejected alternatives and a dependency graph. No implementation begins until the user explicitly approves the architecture and first implementation batch.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R20], [R22], [R23], [R24]

ENGINEERING NOTE 107 |

Phase 2: source inventory and fixtures

Purpose and product decision. Create the data-source catalogue, a verified/blocked/pending matrix and a fixture-only ingestion path before depending on live services. This phase enables repeatable development even when websites are unavailable.

Implementation contract. Add source metadata, sample fixtures with licensing/provenance notes, import adapters and tests. Every fixture is synthetic or permission-cleared and contains representative timestamps, spatial fields, quality failures and source changes.

Failure modes and evidence constraints. Do not scrape websites at scale or make hidden network calls. Do not mark a source “verified” until current access and content have actually been checked.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 2: source inventory and fixtures. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. User can import a supplied sample, see source status and data quality and reproduce the same canonical rows offline. No paid key is required.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2], [R4], [R5]

ENGINEERING NOTE 108 |

Phase 3: schema and ingestion

Purpose and product decision. Build the canonical schema, migrations, import transaction, row validation, quarantine flow and idempotent reprocessing. Keep raw source lineage and quality flags.

Implementation contract. Implement one reliable CSV adapter and one geospatial file adapter. Document the data dictionary, IDs, temporal rules, geometry CRS, null semantics and schema migration contract.

Failure modes and evidence constraints. Don't add every connector at once. Premature adapters multiply unverified assumptions and make debugging impossible.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 3: schema and ingestion. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A clean DB can migrate from empty to current, same file re-imports idempotently and malformed rows are quarantined with actionable feedback.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2], [R22], [R23]

ENGINEERING NOTE 109 |

Phase 4: deterministic analytics

Purpose and product decision. Build dashboard metrics using tested backend logic and SQL. This is the first complete decision-support slice and the baseline to beat for AI features.

Implementation contract. Implement counts, trends, category/area filters, hotspot definitions and related-report explorer. Every aggregate includes date range, record count, exclusions and a drill-through list.

Failure modes and evidence constraints. The dataset may not support trustworthy area comparisons or trends. UI should offer descriptive analytics even if forecasting is disabled.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 4: deterministic analytics. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Hand-calculated fixture outputs pass, report links open source records, and empty/missing data are not displayed as zero incidents.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R17], [R21]

ENGINEERING NOTE 110 |

Phase 5: mapping and geospatial quality

Purpose and product decision. Add maps only after geography and CRS are explicit. Map and analytics views must use the same data filters and report uncertainty in location.

Implementation contract. Implement map layer, marker clusters, time/category filters, ward/grid rollups if boundaries are verified and a fallback table. Add geometry checks and configurable boundary data.

Failure modes and evidence constraints. Do not invent administrative boundaries or portray uncertain geocodes as precise. Avoid dependence on public tile servers for offline tests.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 5: mapping and geospatial quality. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Known fixture coordinates appear correctly, filters match list results and app remains usable without network tiles.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R11], [R12], [R22], [R23]

ENGINEERING NOTE 111 |

Phase 6: ML baselines

Purpose and product decision. Select one ML task based on available labels. Train the simplest sensible baseline, compare to a trivial baseline and write an evaluation report before building a more complex model.

Implementation contract. Candidate priority: category classifier if human labels exist; duplicate candidate ranking if reviewer labels can be collected; report-volume forecasting only if time history is sufficient. Keep experiments out of core import and UI logic.

Failure modes and evidence constraints. If there are no reliable labels, do not fabricate accuracy. Use deterministic descriptive analytics and create a labelled-data collection plan instead.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 6: ml baselines. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The model beats the baseline on a properly held-out evaluation or remains clearly experimental/off by default. Artifacts include a model card, confusion/errors and reproduction command.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R21], [R24]

ENGINEERING NOTE 112 |

Phase 7: NLP enhancements

Purpose and product decision. Add text normalization, language analysis, place-entity candidates and duplicate suggestions only as a coherent feature. Track provenance and allow humans to correct outputs.

Implementation contract. Start with rules/TF-IDF and local embeddings as an optional layer. Cache model outputs by text/model version, redact before indexing and include confidence/alternative suggestions.

Failure modes and evidence constraints. Do not put all text into a remote API or assume one language. Misclassification must not hide reports from the canonical store.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 7: nlp enhancements. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Manual labelled evaluation exists for each enabled NLP task; low-confidence outcomes are shown as review candidates, not confirmed facts.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R24]

ENGINEERING NOTE 113 |

Phase 8: RAG knowledge base

Purpose and product decision. Build cited retrieval over a small set of approved policy/disaster documents and verified historical summaries. Keep raw documents and index metadata traceable.

Implementation contract. Implement document registry, extraction, chunking, local retrieval, answer schema, citation validator, source date display and an unanswerable-question benchmark.

Failure modes and evidence constraints. RAG does not make out-of-date or vague documents authoritative. Keep data queries separate from document retrieval and label general guidance distinct from area-specific evidence.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 8: rag knowledge base. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Answerable questions cite the right page/section; unanswerable/conflicting questions abstain; model-disabled core product still works.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R6], [R7], [R16], [R24]

ENGINEERING NOTE 114 |

Phase 9: bounded agent workflow

Purpose and product decision. Create one agent workflow for “investigate this area and draft a brief.” It should call a limited set of typed tools and return a reviewed evidence bundle.

Implementation contract. Implement a deterministic state machine or minimal orchestrator with scope validation, allowlisted SQL/analytics tools, maximum steps, timeout, structured output, audit events and human review.

Failure modes and evidence constraints. No shell execution, arbitrary SQL, arbitrary URLs, municipal submissions, emergency alerts or direct action on people. Prompts are not a security boundary.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 9: bounded agent workflow. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Adversarial tests show tools block prompt injection, SQL write attempts and scope expansion. The final draft includes source citations, coverage caveats and review state.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19], [R20]

ENGINEERING NOTE 115 |

Phase 10: UX and exports

Purpose and product decision. Finish the main analyst journey: import data, verify source and quality, inspect map/trends, investigate related reports, search guidance, draft an evidence brief and review/export it.

Implementation contract. Use a consistent filter state and a typed report object. Provide CSV/JSON/HTML/PDF export if feasible, avoiding sensitive fields and adding source attribution and generated-on time.

Failure modes and evidence constraints. A visually polished dashboard with dead buttons or untraceable values is not release-ready. Avoid adding a feature just to show another library.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 10: ux and exports. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The core workflow passes end-to-end tests and a first-time user can complete it without CLI interventions after startup.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R16], [R17]

ENGINEERING NOTE 116 |

Phase 11: hardening and evaluation

Purpose and product decision. Before the MVP pilot, verify core correctness, security controls, data licences, privacy redaction, failure states and documented model limits.

Implementation contract. Run unit/contract/integration/end-to-end tests, route security review, import adversarial tests, source attribution check, RAG benchmark, model evaluation and visual accessibility pass. Tag every known risk with severity and owner.

Failure modes and evidence constraints. A passing suite does not prove real-world effectiveness. Separate technical release readiness from validated usefulness with a human pilot.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 11: hardening and evaluation. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. All critical/high issues are fixed or explicitly block public exposure. Open limitations appear in documentation and in the UI where needed.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19], [R20], [R21]

ENGINEERING NOTE 117 |

Phase 12: pilot and decision

Purpose and product decision. The pilot should answer whether CityPulse helps a real analyst investigate recurrence and prepare a better evidence brief. It must also reveal data access and trust barriers before wider scope.

Implementation contract. Choose one area, date range, issue type and user task. Use real data only with permission/terms satisfied; otherwise use a clearly identified historical/public case study or synthetic demo. Capture pre/post task duration and correction quality.

Failure modes and evidence constraints. One successful demo is not proof of reliable city-scale prediction or commercial viability. A negative pilot result is valuable evidence for changing the wedge.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 12: pilot and decision. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Publish a pilot memo with methodology, participant context, source caveats, measured results, errors, incidents, limitations and a go/no-go recommendation.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R16], [R17], [R18]

ENGINEERING NOTE 118 |

Phase 13: roadmap exit criteria

Purpose and product decision. Phases should gate later complexity. Each gate asks if the product foundation is stable and whether evidence supports advancing—not whether the calendar says the phase is over.

Implementation contract. Use checklists for build reproducibility, database migrations, tests, source compliance, evaluation, documented tradeoffs, user review and no hidden paid API use. Include a separate acceptance statement for every model-enabled capability.

Failure modes and evidence constraints. Do not declare an MVP done just because the UI loads or an LLM answers. For every core claim, distinguish implemented, tested, measured and externally validated.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to phase 13: roadmap exit criteria. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A phase closes only when artifacts and test evidence are linked in the change summary; the developer/user authorizes the next batch.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R18], [R19]

ENGINEERING NOTE 119 |

Claude Code working agreement

Purpose and product decision. Claude Code Pro is the development agent, not the product runtime model. Its job is to inspect, propose, implement small batches and explain decisions. The developer remains in control of repository changes and Git history.

Implementation contract. Every prompt limits work to three or four related tasks. The response must explain why the change was made, how it works, exact files changed, commands/tests run, failures, risks and suggested next work. Show Git commands only; never commit or push automatically.

Failure modes and evidence constraints. Do not ask an agent to “build the whole app” or to continue across unreviewed phase gates. It may introduce dependencies, reshape files or use APIs that break the zero-cost objective.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to claude code working agreement. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The working agreement is placed in project guidance after repository audit and user approval. All prompts repeat no paid runtime API, no commits/pushes and stop-at-gate rules.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 120 |

README and project guidance

Purpose and product decision. The repository should explain what the product does, how to run locally, source/licence status, data modes, architecture, model capabilities and limitations. The specific guidance file name should fit what already exists.

Implementation contract. After audit, update an existing CLAUDE.md or create one only with approval. Include project mission, stack decisions, code architecture, test commands, data contracts, security rules, “never do” list, phase plan and how to report completion.

Failure modes and evidence constraints. Don't overwrite existing project knowledge or create competing guidance files. First inspect existing README and agent instructions.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to readme and project guidance. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A new contributor can locate setup commands, architecture decisions, first task and quality gates quickly; docs agree with actual commands and current code.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 121 |

Prompt design for implementation batches

Purpose and product decision. A good implementation prompt has a narrow outcome, constraints, files/components in scope, acceptance criteria, tests, expected summary and explicit stop condition. It should ask for the smallest vertical slice rather than a theoretical architecture rewrite.

Implementation contract. Use a repeated prompt template: Context, verified repository facts, goal, in-scope tasks (max 3-4), out-of-scope, constraints, implementation notes, acceptance tests, reporting requirements and stop point. Feed the previous batch summary into the next prompt.

Failure modes and evidence constraints. If scope is ambiguous, Claude may infer more than requested. Keep the task narrow and direct it to ask before irreversible or broad changes.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to prompt design for implementation batches. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Every completed batch corresponds to a reviewable diff and exact test evidence. Git remains under user control, and no phase advances automatically.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 122 |

Code review and phase gate

Purpose and product decision. Each batch ends with review, not another implementation request. A phase gate checks expected files, acceptance tests, security, data quality and documentation, then produces pass/fail with evidence.

Implementation contract. Review diffs, run target tests, inspect dependency changes, check secrets and licences, verify no remote inference calls were introduced, and compare outputs to fixtures. Record remaining issues by severity.

Failure modes and evidence constraints. Do not accept a vague “all done” summary without commands and results. Do not ask Claude to grade its own code solely through natural-language confidence.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to code review and phase gate. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The gate includes explicit PASS/FAIL/NOT TESTED for each criterion, exact output and a stop until user approval.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 123 |

Cost governance and resource budget

Purpose and product decision. Zero paid API calls is a constraint that needs a test, not only a note. Runtime software should not require a vendor key or make hidden billable calls.

Implementation contract. Maintain a cost/dependency register with runtime API calls, model downloads, storage, compute, hosting, map services and optional accounts. Make external integrations visible and disable them by default unless verified.

Failure modes and evidence constraints. Free services may have rate limits, terms changes or require registration. Local models have no per-call bill but use hardware, power, memory and disk.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to cost governance and resource budget. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. An offline test and code search verify no mandatory hosted model endpoint. The README documents costs that are zero, optional and potentially variable.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R12], [R13], [R24]

ENGINEERING NOTE 124 |

Research uncertainty and source refresh

Purpose and product decision. This document captures a research snapshot on 13 October 2026, but data catalogues, API docs, laws, model licences and service policies change. Source verification is a release task, not a one-time claim.

Implementation contract. Keep a source bibliography and review date; before the first connector implementation, re-open the official page, inspect one real resource and note observed fields and timestamps. Recheck data-protection and licence guidance before any real pilot.

Failure modes and evidence constraints. Search result snippets are not a substitute for reading exact dataset metadata or licence. When a source cannot be opened, record the limitation transparently.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to research uncertainty and source refresh. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Every source in the implemented release is status-labelled and has current review notes. Unverified sources remain candidates and do not appear as active integrations.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2], [R5], [R20]

ENGINEERING NOTE 125 |

Reference architecture and data flow

Purpose and product decision. The default conceptual flow is sources and user uploads, through immutable intake and validation, into canonical SQL records and geospatial features, then deterministic analytics and optional ML/RAG, finally into a human-reviewed evidence brief. Exact components are selected only after repository audit.

Implementation contract. Document the flow as: Source Registry -> Fetch/Upload -> Raw Snapshot -> Parser/Staging -> Quality Gate -> Canonical Store -> Feature/Aggregate Services -> Map/Dashboard -> RAG/Agent Evidence Bundle -> Draft -> Human Review -> Export. The agent calls services, never bypasses them to access raw files or unrestricted databases.

Failure modes and evidence constraints. Do not deploy a large collection of microservices for a single-developer MVP. A modular monolith may be easier to test and run locally; split components only when required by load or trust boundaries.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to reference architecture and data flow. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The architecture page names data contracts and failure handling between each step. Every output is reproducible from source IDs, selected filters, code/model version and run ID.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20], [R22], [R23]

ENGINEERING NOTE 126 |

Concrete database schema decision guide

Purpose and product decision. The database model must preserve source records and build a compact queryable domain model. Table names in this blueprint are proposals, not commands to redesign an existing schema before audit.

Implementation contract. Minimum conceptual tables: `source_registry`; `ingestion_batch`; `raw_object_ref`; `observation`; `area`; `boundary_version`; `document`; `document_chunk`; `analysis_run`; `metric_result`; `duplicate_candidate`; `model_run`; `recommendation_draft`; `review_event`; `audit_event`. A spatial extension may be added only if it simplifies required queries enough to justify local setup.

Failure modes and evidence constraints. Do not store everything in a single unvalidated JSON column or create redundant denormalized copies with conflicting meaning. Use JSON only for source-specific payloads that need preservation, not as the sole canonical contract.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to concrete database schema decision guide. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Review primary keys, uniqueness, foreign keys, nullability, retention and indexes with test fixtures. Confirm the smallest design supports source drill-through, trends, spatial filter and audit before expanding it.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R23]

ENGINEERING NOTE 127 |

Suggested observation record contract

Purpose and product decision. An observation row represents a reported or measured record that came from a specific source. It is distinct from an incident cluster, an official warning, a model prediction and a recommendation.

Implementation contract. Minimum fields: `observation_id`, `source_id`, `batch_id`, `external_id`, `category`, `text_raw_ref` or permitted raw text, `text_normalized`, `reported_at`, `observed_at`, `event_time_precision`, `status_source`, `latitude`, `longitude` or `geometry`, `area_id`, `location_method`, `location_confidence`, `quality_flags`, `created_at`, `record_hash` and `schema_version`. Field semantics must be defined in the data dictionary.

Failure modes and evidence constraints. Do not manufacture exact time or coordinates to fill blanks. If text has personal data, raw storage and indexing must follow privacy/retention rules. Model labels and scores belong in separate versioned tables.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to suggested observation record contract. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. A data-contract test checks mandatory lineage, coordinate bounds, supported category, time semantics and non-destructive raw/normalized separation.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R2], [R14]

ENGINEERING NOTE 128 |

Document, chunk and model-run contracts

Purpose and product decision. Knowledge and ML records require versioning so that citations and scores are reproducible. An embedding alone is not a document and a generated answer alone is not a model evaluation.

Implementation contract. A document record includes source, title, licence, publication/version date, downloaded date, content hash and sensitivity. A chunk record includes document ID, page/section, chunker version, embedding model/version and index ID. A model run includes task, data split, training data version, code commit or version label, parameters, metrics, artifact path and limitations.

Failure modes and evidence constraints. Do not put all text chunk content into logs; document storage and retrieval index have different retention and access considerations. A model result must not be rewritten in place after a model upgrade.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to document, chunk and model-run contracts. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Rebuilding an index records a new version; previous answer traces still reference the evidence actually used. Model cards link to the model run and the exact evaluation data version.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R24]

ENGINEERING NOTE 129 |

API endpoint catalogue for an MVP

Purpose and product decision. API routes should be grouped by domain and expose stable typed results. Framework choice remains open until audit, but endpoint semantics should be defined independently of UI widgets.

Implementation contract. Candidate domains: `GET /health`; `GET /sources`; `GET /sources/{id}/status`; `POST /imports/preview`; `POST /imports`; `GET /observations`; `GET /analytics/summary`; `GET /analytics/trends`; `GET /analytics/hotspots`; `GET /areas`; `GET /documents/search`; `POST /analysis-runs`; `GET /analysis-runs/{id}`; `POST /reports/drafts`; `POST /reports/{id}/review`. Use versioning discipline if external clients will depend on it.

Failure modes and evidence constraints. Do not expose an endpoint that accepts arbitrary SQL, file-system paths or arbitrary external URL. Keep imports size-limited and protect durable mutations where multiple users exist.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to api endpoint catalogue for an mvp. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. An API contract document includes request fields, response examples, query limits, authorization assumptions, error codes and whether operation mutates state. Contract tests validate each implemented route.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R20]

ENGINEERING NOTE 130 |

API errors, pagination and consistency

Purpose and product decision. A stable API tells the UI what happened without leaking internal stack traces. Pagination and limits protect against large source data and accidentally expensive analytical requests.

Implementation contract. Use consistent error shapes with machine-readable code, safe message, request/correlation ID and field-level validation issues when appropriate. Add deterministic ordering and bounded page size. Analysis responses include filters, observation window, source freshness and quality caveats.

Failure modes and evidence constraints. An empty valid result, a source outage and a parser failure must not look the same. A long-running analysis needs cancellation/state rather than a synchronous request that silently times out.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to api errors, pagination and consistency. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. Test invalid filters, oversized pages, bad IDs, missing source, empty dataset, timeout and internal exception sanitization.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R20]

ENGINEERING NOTE 131 |

Source register starting matrix

Purpose and product decision. The source inventory starts with candidates, not confirmed integrations. Prioritize by expected utility, licence clarity and ease of reproducible access.

Implementation contract. Candidate portfolio: data.gov.in catalogue (discovery); Smart Cities Mission portal (catalogue, participating city only); IMD public API reference (weather/rainfall/warnings); MCGM disaster management plan (document/map context); NASA GPM IMERG (regional rainfall context); OpenStreetMap extract (base geography); Copernicus Sentinel (optional remote-sensing research); manual CSV/GeoJSON import; consented partner exports; synthetic fixtures.

Failure modes and evidence constraints. Coverage, current fields, freshness and access must be validated for the chosen ward. A source can be useful for static context but inappropriate for live operational use.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to source register starting matrix. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The actual source register records `candidate`, `verified`, `blocked`, `research-only` or `disabled`, with evidence and date. Only verified resources feed real-data demo paths.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R1], [R4], [R5], [R6], [R8], [R9], [R11], [R13]

ENGINEERING NOTE 132 |

Technical decision records template

Purpose and product decision. Stack and architecture decisions should be written as records with context, decision, alternatives, consequences and review triggers. This keeps Claude Code from repeatedly re-litigating a settled choice or making silent framework changes.

Implementation contract. For every decision capture driver(s), options considered, compatibility with current repo, zero-cost compliance, M1 performance, licence, developer burden, tests, rollback and date. Mark tentative decisions as proposed until the user approves.

Failure modes and evidence constraints. A good decision record cannot be reverse-engineered from a dependency file alone. Existing repo constraints may change the recommendation; the audit is the source of truth.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to technical decision records template. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. No major database, UI framework, vector store or agent library is added without a short decision record and user approval at the phase gate.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

ENGINEERING NOTE 133 |

Definition-of-done rubric

Purpose and product decision. A phase is complete when the requested behaviour works, is tested, documented, reproducible and compatible with constraints. Merely creating files does not meet definition of done.

Implementation contract. Score each criterion as PASS, FAIL or NOT TESTED. Link each pass to an automated test, command output, screenshot, source proof or review record. Explicitly state whether real data, synthetic fixtures or mocks were used.

Failure modes and evidence constraints. Do not interpret skipped tests as passed. Do not claim accuracy or public usefulness based only on a working local demo. A severe risk can keep the phase blocked even if most criteria pass.

Engineering sequence. First, confirm the source/input contract and identify the owner of each field relevant to definition-of-done rubric. Next, isolate domain logic from the user interface and external access, so the same transformation can be tested against a saved fixture. Then persist the result with source lineage, processing version and a quality state. Finally, expose a reviewable output with the scope, assumptions and limitations visible. If the repository already has an equivalent capability, extend it rather than duplicating it.

Required failure behaviour. Handle empty inputs, malformed fields, unknown categories, stale data, partial source outages and disabled optional models explicitly. An unavailable result must remain “unknown / unavailable” rather than being converted to zero, a safe condition or a confident narrative. Any manual override must be attributable and reversible. Generated text may explain stored results but must not silently change source facts.

Definition of done. The gate summary reports failures, untested items, known issues, changed files and next action, then waits for user permission to advance.

Verification checklist

- Unit tests cover normal, boundary, missing-data and malformed-input cases; fixtures are synthetic or permission-cleared.
- An integration/contract test proves that persisted results can be traced to the source and that repeated runs are deterministic where expected.
- The interface or report states the geography, time window, data mode, source freshness and material limitations.
- The no-paid-API path works; optional model/network components may be disabled without breaking the core workflow.

Claude Code handoff. After the read-only audit and architecture approval, implement this as one small vertical slice (no more than 3-4 related tasks). Report why the change was necessary, how it works, exact files changed, commands/tests executed, observed outputs, unresolved issues and the suggested next slice. Do not commit, push, install optional heavy models or advance to another phase automatically.

Related research: [R19], [R20]

Appendix A — Data source verification register template

Field	Required contents
source_id	Stable short identifier used by all imports and citations.
title / owner	Exact dataset/resource title and publishing department or operator.
canonical_url	Exact resource URL, not only a portal home page.
checked_at / checker	Date of current verification and human/automated process used.
access method	Direct download, documented public API, manual export, user-provided file or blocked.
sample proof	File hash, example fields, observed record count/date range or reason no sample could be obtained.
licence and attribution	Exact licence metadata URL, attribution string, third-party restrictions and decision.
coverage	Geography, admin level, spatial resolution, date range, temporal frequency and known gaps.
data quality	Missingness, coordinate accuracy, delay, duplicate behavior, schema stability, representativeness.
status	Candidate / verified / research-only / blocked / disabled, with a plain-language rationale.
recheck trigger	Date or change event that should cause the source to be re-verified.

Source status means only what was tested. “Verified” means that the exact resource could be accessed, its contents and licence were inspected, a sample was parsed, and the limitations were recorded. It does not mean the source is complete, accurate, authoritative for every use, or guaranteed to remain online.

Appendix B — Proposed entity relationship outline

```
source_registry (1) ---- (many) ingestion_batch
ingestion_batch (1) ---- (many) observation
source_registry (1) ---- (many) source_snapshot
area (1) ---- (many) observation [only when area match is known]
boundary_version (1) ---- (many) area geometry versions
source_registry (1) ---- (many) document
document (1) ---- (many) document_chunk
analysis_run (1) ---- (many) metric_result
observation (many) ---- (many) duplicate_candidate edges (not destructive merge)
model_run (1) ---- (many) model output references
analysis_run + document_chunk IDs ---- (many) recommendation_draft
recommendation_draft (1) ---- (many) review_event
all material changes ---- audit_event
```

Use foreign keys wherever the database supports them; index source ID, external ID, timestamps, category and commonly filtered area. Add geometry indexes only if using a spatial database and after measuring queries. The relational model must preserve original observation records even when analysts group possible duplicates into one reviewed incident cluster. Avoid coupling model-run schema to a specific ML library.

Appendix C — Evaluation metric menu

Capability	Baseline / metric	Required caveat
Import	Accepted/quarantined rows, validation error counts, repeated import idempotence	A successful parse does not prove source accuracy.
Geocoding	Precision on manually verified sample, unresolved rate, error distance	Report coordinate granularity and boundary vintage.
Hotspots	Comparison to simple count/grid baseline; stability under parameter change	A hotspot is a statistic over chosen filters, not a ground truth ranking.
Category NLP	Macro-F1, per-class precision/recall, confusion matrix	Label quality and language coverage may be low.
Duplicate candidate	Precision@K, recall on reviewed pairs, reviewer acceptance	Suggestions must not auto-delete or suppress issues.
Report-volume forecast	MAE/RMSE vs naive and seasonal-naive baselines, chronological backtests	Counts are reports, not necessarily physical incidents.
RAG retrieval	Recall@K, citation precision, correct abstention rate	Evaluation set needs unanswerable and conflicting-source questions.
RAG answer	Evidence support rate, citation correctness, reviewer edit rate	Automated judging is supplementary, not sole ground truth.
Agent	Successful bounded task rate, unsafe tool-block rate, steps/latency, replay success	Never reward task completion that violates a safety constraint.
Product	Time-to-brief, correction rate, source-link validity, task completion	Use real user feedback; synthetic demo is not external validation.

For any forecasting task, use chronological splits that evaluate the future relative to the training window; scikit-learn documents why random splits are unsuitable for autocorrelated time series. [R21] Keep the split logic in tested code and store the exact data cutoff.

Appendix D — Security and safety release blockers

Risk	Control required for MVP	Release stance
Prompt injection in reports/docs	Retrieved content is untrusted; tools independently authorize; adversarial tests exist.	Block agent release if it can expand tool permissions.
Arbitrary SQL or data exposure	Allowlisted analytical tools, read-only DB role, parameterized queries and row/time limits.	Block agent mode if arbitrary SQL executes.
Unsafe URL / SSRF	No arbitrary LLM URLs; host allowlist; redirects validated; private ranges denied.	Disable external fetch when uncertain.
Sensitive data in embeddings/logs	Redact/minimize fields, retention plan, no raw prompt logs by default.	Use synthetic data for public demos.
Source licence unclear	Source status marked pending or blocked; attribution tracked.	Do not redistribute unclear data.
False street-level hazard claim	Resolution and validation limits visible; no emergency warnings.	Block predictive flood claims without valid labels.
Live source outage / stale data	Staleness visible; no-data not interpreted as no-incident.	Degrade visibly, not silently.
Malicious/oversized import	Size/time/count limits, safe temp files, parser tests, transaction rollback.	Reject unsafe files.
Public service overuse	Adhere to API/tile policies, use extracts/fixtures, conservative retries.	No bulk scraping or tile prefetch.
Unreviewed recommendations	Draft-only workflow and human review event.	No external action channel in MVP.

This checklist is based on general trustworthy-AI and API-security principles; it is not a legal certification or a penetration test. Specific risks and applicable rules must be revisited before any real civic deployment. [R18-R20]

Appendix E — Claude Code prompt 0: read-only repository audit

You are starting work on CityPulse AI. YOUR ONLY TASK IS STRICTLY READ-ONLY REPOSITORY RECONNAISSANCE. Do not implement anything. Do not create, edit, delete, move, rename, format or generate files. Do not install or update dependencies. Do not run migrations. Do not checkout, reset, stash, commit or push. Do not execute commands whose side effects are uncertain. If this repository is not accessible, report that fact and stop; do not guess or switch to another project.

Inspect: current working directory and top-level tree; Git branch/status and uncommitted work; README and any CLAUDE.md/agent instructions; package manifests and lockfiles; app entry points; route/controller/service boundaries; schema and migrations; import/parsing/data contracts; tests and test configuration; analytics/ML/NLP/RAG/agent modules; environment/config patterns; sample datasets and licences; CI/build/deploy scripts; known TODOs. Use only safe read commands. Do not assume a ProblemSpec or DataPilot foundation exists unless you find exact files and explain them.

Return a structured report: (1) verified repository facts with exact paths; (2) implemented/partial/missing/unknown feature matrix; (3) current stack and why evidenced by files; (4) test commands and results, distinguishing not run from failed; (5) data/source/licence status; (6) security and maintainability risks ranked Critical/High/Medium/Low; (7) constraints and likely blockers for a zero-paid-runtime-API local MVP on a MacBook Air M1; (8) two or more stack/architecture options, without choosing or installing one; (9) a phased proposal with tests and acceptance criteria.

At the end, state whether you changed anything (must be no), list commands used, identify uncertainties, and STOP. Ask for my approval before any edits or dependency changes.

Appendix F — Claude Code prompt 1: architecture and stack proposal

Use only the verified findings from the approved read-only audit and this CityPulse blueprint. Do not change files yet. Recommend the implementation architecture and technology stack for a student-built, local-first CityPulse AI MVP on an Apple Silicon MacBook Air with no mandatory paid runtime APIs or cloud credits.

Compare at least two reasonable options for UI, backend/API, relational storage/spatial analysis, ingestion/validation, local document retrieval/embeddings, optional local LLM runtime, test stack and packaging. For each option score: compatibility with existing code, zero-paid-runtime-API compliance, licence clarity, local M1 performance, installation complexity, developer learning cost, testability, data integrity, maintainability and deployment route. Prefer minimal complexity; do not add an agent framework, vector database, spatial database or frontend framework unless the benefit is concrete. Existing working code is a constraint and must not be rewritten without clear reason.

Deliver: context diagram and data flow; proposed modules/contracts; entity relationship model; route/API catalogue; source/data-mode design; security boundaries; data validation and evaluation strategy; dependency/licence/cost register; risks and rejected alternatives; phase 0-13 mapping to existing vs missing code; first implementation batch of only 3-4 related tasks with tests. Clearly separate facts from assumptions. Do not install dependencies or modify files. End with a precise approval request and STOP until I respond.

Appendix G — Claude Code prompt 2: implement one small batch

Implement ONLY the following user-approved batch: [paste 3-4 related tasks]. Use the approved architecture decision records and the current repository state. Before editing, briefly restate intended files and tests. Preserve unrelated work and existing conventions. Do not perform broad refactors or silently change the stack. Do not install unrelated dependencies, use paid inference APIs, create cloud dependencies, commit or push. If a requirement conflicts with existing code or the blueprint, stop and explain before making a broad change.

Implementation expectations: add or update focused tests with each behavior; preserve source provenance and quality flags; validate external inputs; keep model calls optional and local; use typed tool contracts and parameterized SQL; log safely without personal data; document any changed contract and migration. Use deterministic fixtures rather than real network calls by default.

At completion, report (1) why this work was necessary; (2) how it works; (3) exact files added/changed and key functions/contracts; (4) commands run and actual pass/fail output summary; (5) manual validation performed; (6) failure cases/limitations; (7) risks/dependencies/licence changes; (8) how to undo the batch if required; and (9) suggested next small batch. Do not commit or push. Do not start the next batch. Stop for my review.

Appendix H — Claude Code prompt 3: verify a data source

Investigate only this proposed data resource: [exact URL/title]. Do not implement a connector yet. Verify the exact resource page, publishing owner, file/API access, observed schema, sample timestamps, geographic/time resolution, update behavior, licence/terms, attribution, third-party rights, registration requirements, rate limits and privacy concerns. If you cannot fetch or inspect it, mark each unverified item unknown; do not infer from a search snippet or homepage. Do not bulk-download, scrape tiles or make many repeated calls. Avoid sending personal information.

Return: source card; source status (verified/candidate/research-only/blocked); exact evidence URLs and review date; a small field mapping to the canonical CityPulse schema; data-quality concerns; limitations; recommended attribution text; proposed fixture and contract tests; decision to include/avoid/defer. State whether any action could cause a charge or require an account. Do not create files, call the source on a schedule or modify the repo. Stop for approval.

Appendix I — Claude Code prompt 4: security and correctness review

Perform a review of the current diff and relevant surrounding code only. Do not fix issues yet. Evaluate source provenance and licence handling; file upload parsing and resource limits; SQL injection and read-only agent tools; URL allowlists/SSRF; authorization assumptions; prompt injection from imported/retrieved text; data leakage in logs/embeddings/exports; unsafe defaults; stale-source behavior; geometry/time/data leakage; hidden network or paid API calls; dependency/licence risks; test coverage; and claims in the UI or README that exceed evidence.

Return findings in a table with severity, file/line, demonstrated failure path, likely impact, precondition, recommended fix, regression test and whether it is a release blocker. Separate confirmed findings from hypotheses. Run only safe non-destructive tests that do not expose secrets or modify user data; report tests not run. Do not edit files, install tooling, commit or push. Stop for approval of fixes.

Appendix J — Claude Code prompt 5: phase gate

Evaluate the current phase against its approved acceptance criteria and the CityPulse blueprint. Do not implement new functionality or start the next phase. Review the diff, changed files, test outputs, documentation, dependency/license changes, data provenance, zero-paid-API behavior, security controls and the current limitations. Distinguish PASS, FAIL and NOT TESTED; link every PASS to direct evidence such as a test command/result, fixture output, source metadata or manual test.

Return: acceptance matrix; bugs/risks ranked by severity; tests actually executed and their result; manual test steps; data mode used (synthetic/real/mock); whether the core no-model/offline path still works; licence/source status; performance and accessibility notes; exact files changed; and a recommendation of PASS, PASS WITH EXPLICIT RISKS, or BLOCKED. Do not treat skipped tests as passed. If anything blocks the phase, name the smallest next batch that would fix it. STOP and wait for my explicit approval before proceeding.

Appendix K — Claude Code prompt 6: demo and portfolio honesty check

Review the project as a skeptical technical interviewer and a potential early pilot user. Do not exaggerate. Inspect actual code, reproducible setup, architecture decisions, source register, test outputs, ML/RAG evaluation, security controls, licence/attribution, demo data labels and UI claims. Distinguish: implemented, tested, measured offline, validated with real users, and only proposed.

Produce a concise demo script (5-7 minutes), a repo-readiness checklist, top five strengths with evidence, top five gaps, and recommended evidence to publish (architecture, test summary, model card, data card, source register, screenshots, evaluation tables and known limitations). Flag any claim about predicting floods, municipal adoption, coverage, accuracy, savings or commercial demand that is not supported by evidence. Do not modify code or documentation. Stop for approval.

Appendix L — Glossary

Term	Meaning in CityPulse
Observation	A record reported or measured by a source; not automatically a confirmed physical incident.
Incident cluster	A reviewed grouping of observations that may refer to the same event.
Hotspot	A statistical concentration defined by a specified spatial unit, time window and metric.
Source provenance	Metadata connecting a record or claim to source, version, access time and transformations.
Lineage	The chain connecting input records to canonical rows, derived metrics, model outputs and reports.
CRS	Coordinate reference system used to interpret geometry and distances.
Geocoding	Converting a place/address string into candidate coordinates with a method and uncertainty.
Data freshness	How current a source observation or publication is relative to its expected cadence.
RAG	Retrieval-augmented generation: fetching relevant evidence before drafting an LLM answer.
Embedding	A vector representation used for similarity search; similarity is not proof of identity.
Abstention	A deliberate “insufficient evidence” response rather than a fabricated answer.
Prompt injection	Untrusted content attempting to influence an AI model’s instructions or tool use.
Agent tool	A typed capability an AI workflow may invoke, enforced by application code.
Calibration	Agreement between predicted probabilities and observed frequencies.
Data leakage	Using information unavailable at the intended prediction time or evaluation point.
Idempotency	Repeating an operation with the same input does not create unintended duplicate effects.

Term	Meaning in CityPulse
Human-in-the-loop	A workflow in which a person can inspect, correct, approve or reject outputs.
Source coverage	Where, when and for which categories a data source contains observations.
Synthetic data	Artificial data generated for tests/demos, explicitly not represented as a real observation.
Model card	A concise record of model purpose, data, evaluation, limitations and intended use.

Bibliography and source notes

The sources below were consulted for this blueprint. They are starting points for implementation research, not blanket confirmation that every dataset is downloadable or usable for the proposed pilot. Re-open the exact source resource and its licence before ingestion. Accessed/researched on 14 October 2026 unless the source is explicitly versioned otherwise.

[R1] Open Government Data Platform India, Home / catalogue. https://www.data.gov.in/
Official Indian government catalogue; records may link onward to department-owned data. Catalogue presence is not proof that a needed dataset/API is available.

[R2] Open Government Data Platform India, Terms of Use. https://www.data.gov.in/terms-of-use
The individual dataset record license/metadata matters; verify third-party material and attribution requirements before reuse.

[R3] Government Open Data License - India (GODL-India).
https://data.gov.in/sites/default/files/NDSAP_OpenDataLicense.pdf
Where a dataset is offered under this licence, record attribution and follow the specific licence terms; do not generalize it to every linked dataset.

[R4] Smart Cities Mission Data Portal - cities and datasets. https://smartcities.data.gov.in/cities
Useful candidate catalogue for participating smart cities; do not assume coverage for Mumbai or that every record exposes a stable API.

[R5] India Meteorological Department, Public API Reference. https://api.imd.gov.in/public/api_reference.html
Documents weather forecast, current weather, district warning and rainfall endpoints. Validate availability, data semantics, access conditions and stability before integration.

[R6] MCGM / Mumbai District Disaster Management Plan - City.
https://dm.mcgm.gov.in/assets/pdf/Disaster%20Management%20Plan-%20City.pdf
Potential source for local hazard context and waterlogging-related maps. Document is not automatically a ready-to-use vector layer; manual interpretation/georeferencing may be required.

[R7] NDMA / NITI Aayog-hosted National Guidelines: Management of Urban Flooding.
https://niti.gov.in/sites/default/files/2021-03/Annexure-NDMA-Guidelines-compressed.pdf
Policy and conceptual context for urban flooding; use as guidance, not as a street-level incident feed.

[R8] NASA Global Precipitation Measurement - IMERG data directory. https://gpm.nasa.gov/data/directory
Satellite precipitation data; product versions, latency and resolution vary. IMERG is about 0.1 degree (~10 km), unsuitable as sole street-level flood truth.

[R9] Copernicus data access and terms. https://www.copernicus.eu/en/terms-use/how-access-data
Copernicus data are generally free/full/open, with service and product-specific access steps and exceptions; check the exact product licence.

[R10] Copernicus Sentinel data licence. https://cds.climate.copernicus.eu/licences/ec-sentinel
Sentinel use rights and disclaimers. Remote sensing may support land/urban context but not guarantee observation of a specific local incident.

[R11] OpenStreetMap Copyright and Licence. https://www.openstreetmap.org/copyright
OSM data are under ODbL with attribution and share-alike considerations for derived databases; review licence obligations for the intended use.

[R12] OpenStreetMap Standard Tile Usage Policy. https://operations.osmfoundation.org/policies/tiles/
Community tile servers are best-effort, not a free unlimited tile API; no bulk tile scraping; cache and attribute correctly.

[R13] Geofabrik downloads. https://download.geofabrik.de/
Practical route to OSM region extracts; data remain subject to OSM licensing. Prefer extracts for local analysis over hammering live APIs.

[R14] Open311 API / civic issue tracking standard. https://www.open311.org/api/
Interoperability pattern for civic service requests, not evidence any particular Indian municipal corporation exposes a compatible endpoint.

[R15] FixMyStreet Platform overview. https://fixmystreet.org/overview/
Established open-source geospatial public problem-reporting platform; prior art for reporting and routing; CityPulse should differentiate in evidence, recurrence analytics and explainable decision support.

[R16] UN-Habitat Digital Cities Toolkit. https://unhabitat.org/digitalcitiestoolkit/digital_city_toolkit.html
Useful framing for digital-city governance, public interest, privacy and data responsibility.

[R17] UN-Habitat urban observatory guide. https://unhabitat.org/a-guide-to-setting-up-an-urban-observatory-popular-version
Guidance for urban data observatories and evidence-based local planning.

[R18] OECD, Governing with Artificial Intelligence: Are governments ready? (2024).
https://www.oecd.org/en/publications/governing-with-artificial-intelligence_26324bc2-en.html
Public-sector AI benefits and governance challenges: transparency, accountability, privacy and capability.

[R19] NIST AI Risk Management Framework 1.0. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf
Voluntary framework for context-specific trustworthy AI risk management across design and deployment.

[R20] OWASP API Security Top 10 - 2023. https://api-security.owasp.org/editions/2023/en/0x00-header/
API risk categories include access control, authentication, resource consumption, SSRF, configuration and unsafe API consumption.

[R21] scikit-learn: Time series cross-validation. https://scikit-learn.org/stable/modules/cross_validation.html
Time-ordered evaluation avoids training on future information and evaluating on the past; chronology and leakage controls matter.

[R22] GeoPandas documentation. https://geopandas.org/en/latest/docs.html
Open-source vector data workflows for reading, projection, overlay, spatial joins, mapping and analysis.

[R23] PostGIS official manual. https://postgis.net/documentation/manual/
Spatial types, indexing and SQL functions for relational geospatial analysis when a spatial DB is justified.

[R24] Sentence Transformers documentation. https://www.sbert.net/
Local sentence embeddings for semantic search; select multilingual model based on evidence and local hardware.

[R25] Open Government Data policies. https://www.data.gov.in/policies
National open-data policy and licence links; always preserve dataset-level licence details.

Additional implementation references to review at stack-selection time: official documentation for the frameworks and packages actually present in the repository; current India data protection legislation/rules and the relevant municipal data-sharing terms; and licences of any selected local language model, embedding model or pretrained weights. These must be verified at the time of use. This blueprint is not legal advice.